
第六章：支持向量机

大纲

□ 支持向量机基本型

- 间隔与支持向量
- 支持向量机基本型

□ 对偶问题

□ SMO算法（选讲）

□ 核函数

□ 软间隔与正则化

□ 支持向量回归

复习：线形模型（针对分类问题）

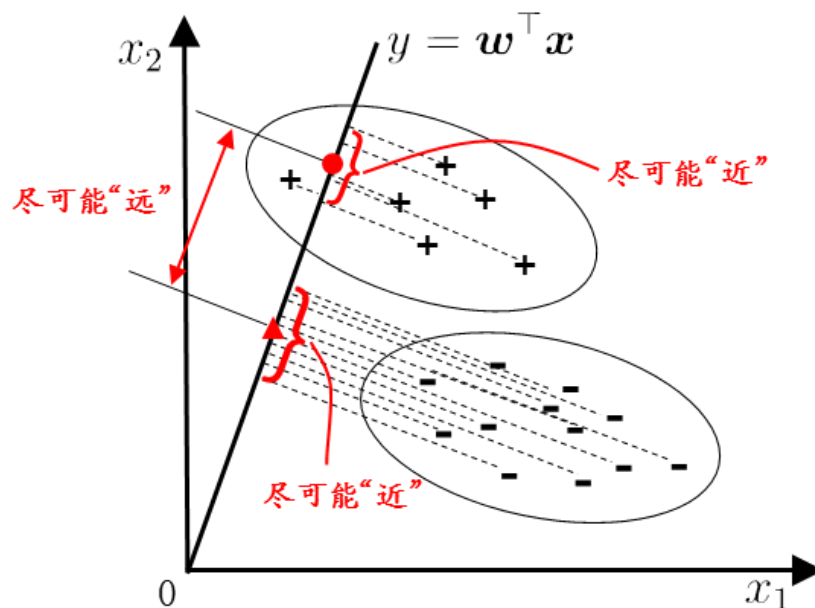
□ 线性判别分析LDA

寻找直线 $f(\mathbf{x}) = \mathbf{w}^T \mathbf{x}$ ，使得当给定的训练样例集投影到直线上时满足

➤ 同类样本的投影点尽可能接近

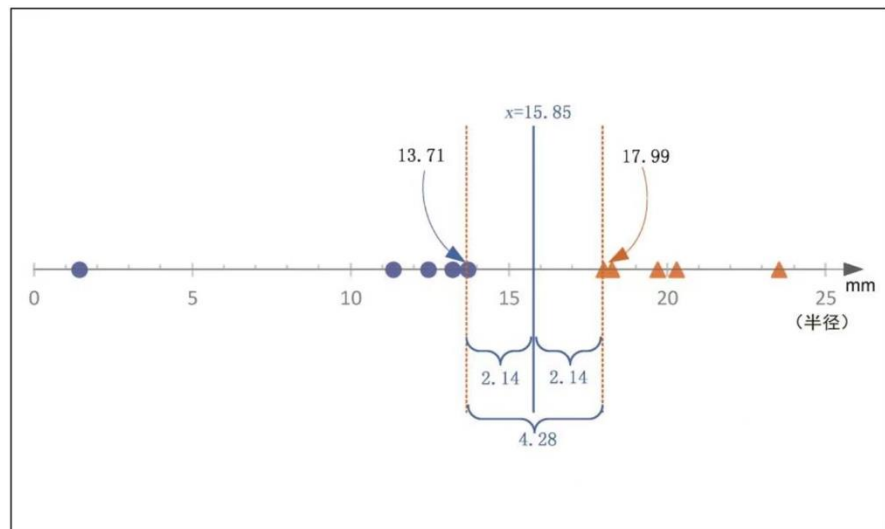
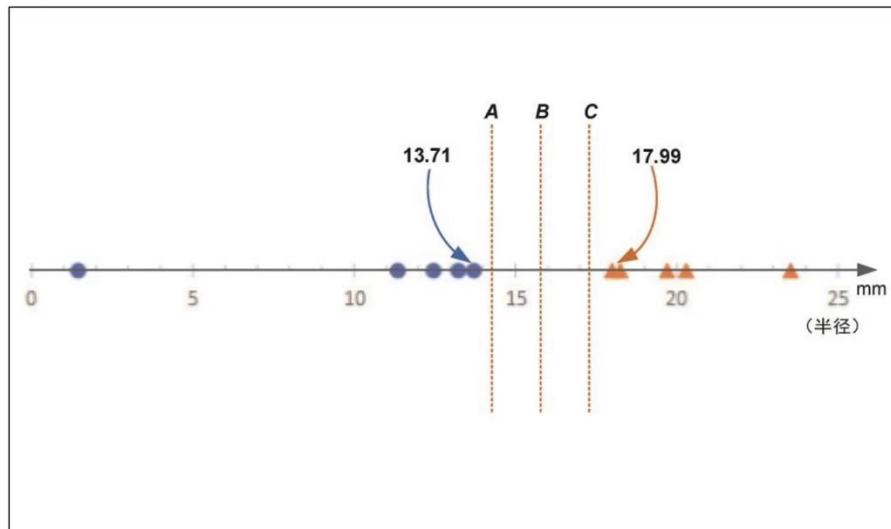
➤ 异类样本的投影点尽可能远离

分类新样本时，可将新样本数据投影到直线上，再根据投影点位置确定新样本的类别



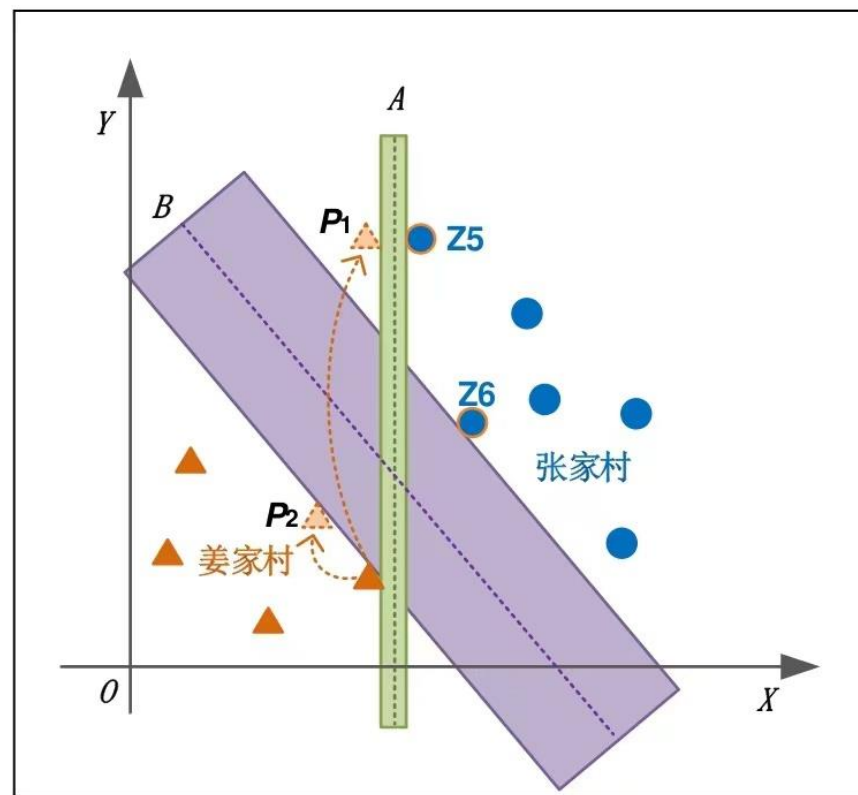
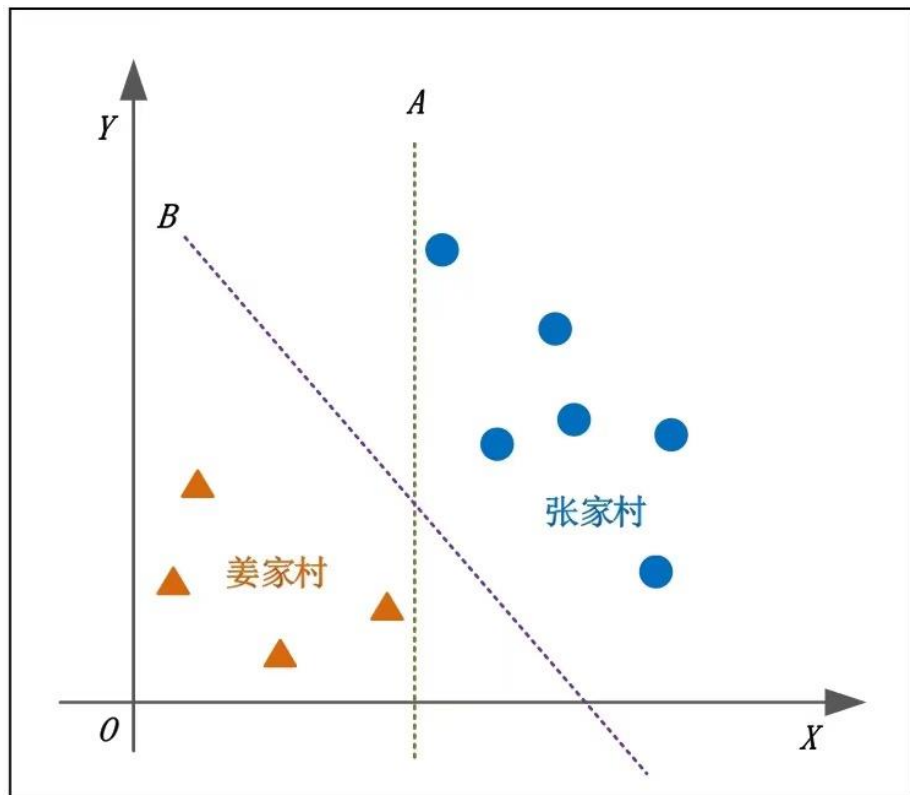
间隔与支持向量

□ 分类学习(一维数据)



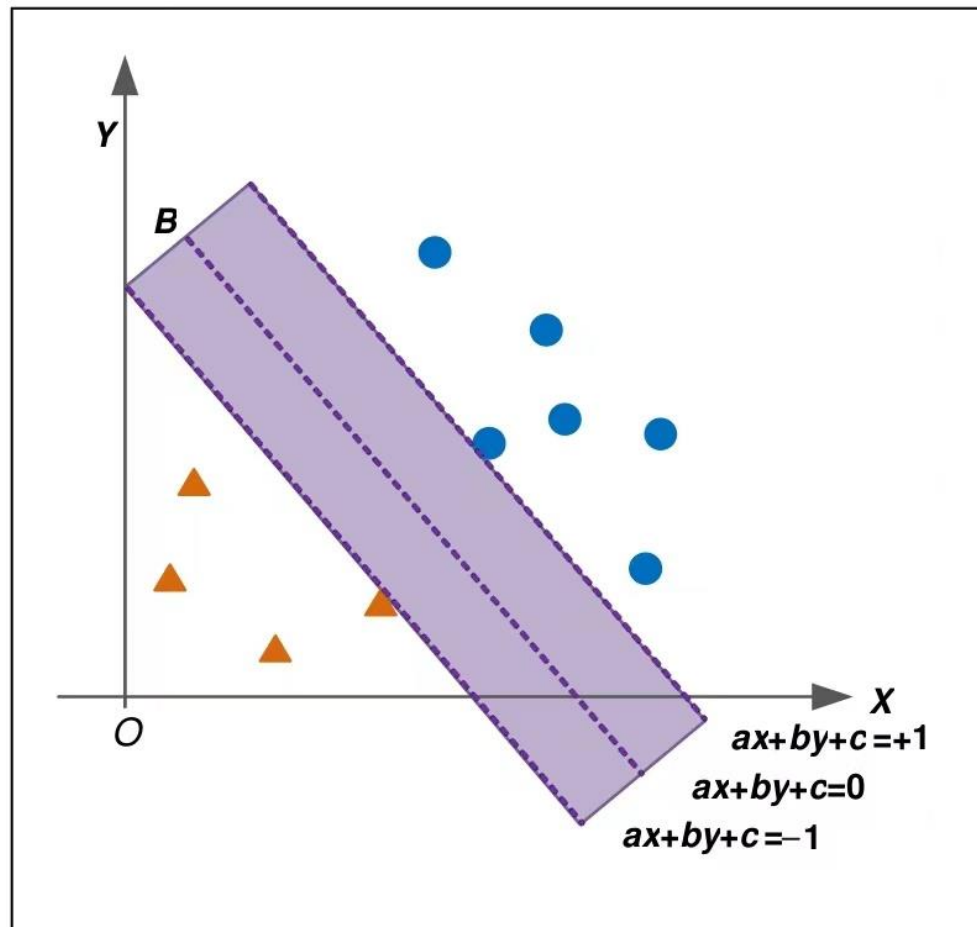
间隔与支持向量

□ 分类学习(二维数据)



间隔与支持向量

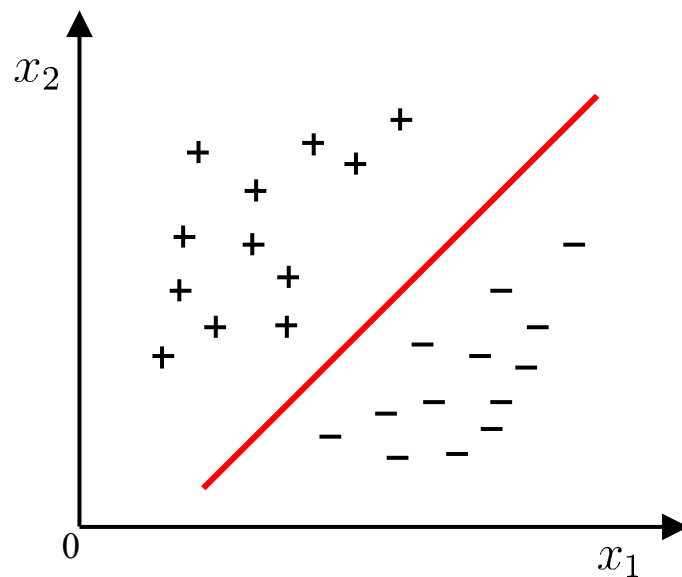
□ 分类学习(二维数据)



间隔与支持向量

□ 分类学习

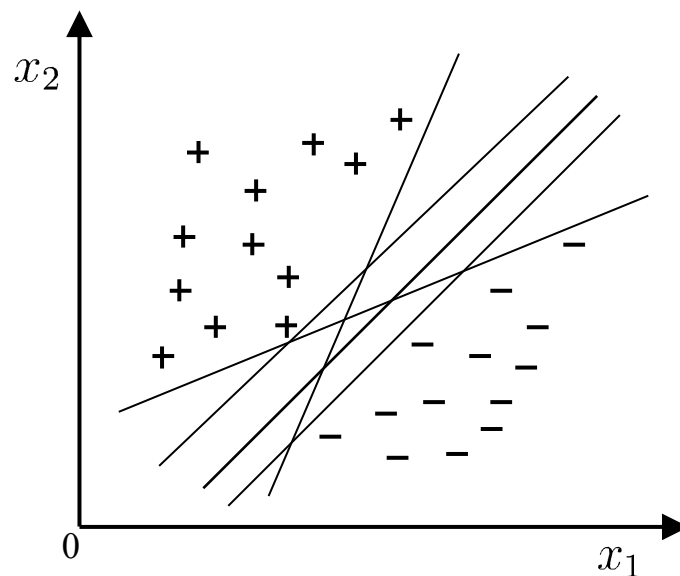
给定数据集 $D = \{(\mathbf{x}_1, y_1); \dots; (\mathbf{x}_m, y_m)\}$, 其中 $\mathbf{x}_i$ 有 d 个维度, $y_i \in \{-1, +1\}$
一种分类基本想法是在训练数据集 D 所处的样本空间中寻找到一个合适的线性超平面, 使其将正、负样本完全分开



间隔与支持向量

□ 分类学习

给定数据集 $D = \{(\mathbf{x}_1, y_1); \dots; (\mathbf{x}_m, y_m)\}$, 其中 $\mathbf{x}_i$ 有 d 个维度, $y_i \in \{-1, +1\}$
一种分类基本想法是在训练数据集 D 所处的样本空间中寻找到一个合适的线性超平面, 使其将正、负样本完全分开

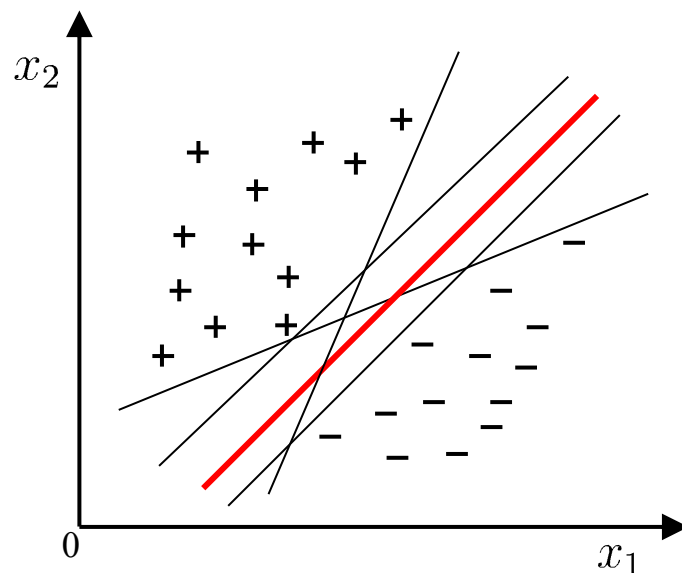


Q: 空间中满足要求的线性超平面可能有多, 选择哪一个最为合适 ?

间隔与支持向量

□ 分类学习

给定数据集 $D = \{(\mathbf{x}_1, y_1); \dots; (\mathbf{x}_m, y_m)\}$, 其中 $\mathbf{x}_i$ 有 d 个维度, $y_i \in \{-1, +1\}$
一种分类基本想法是在训练数据集 D 所处的样本空间中寻找到一个合适的线性超平面, 使其将正、负样本完全分开

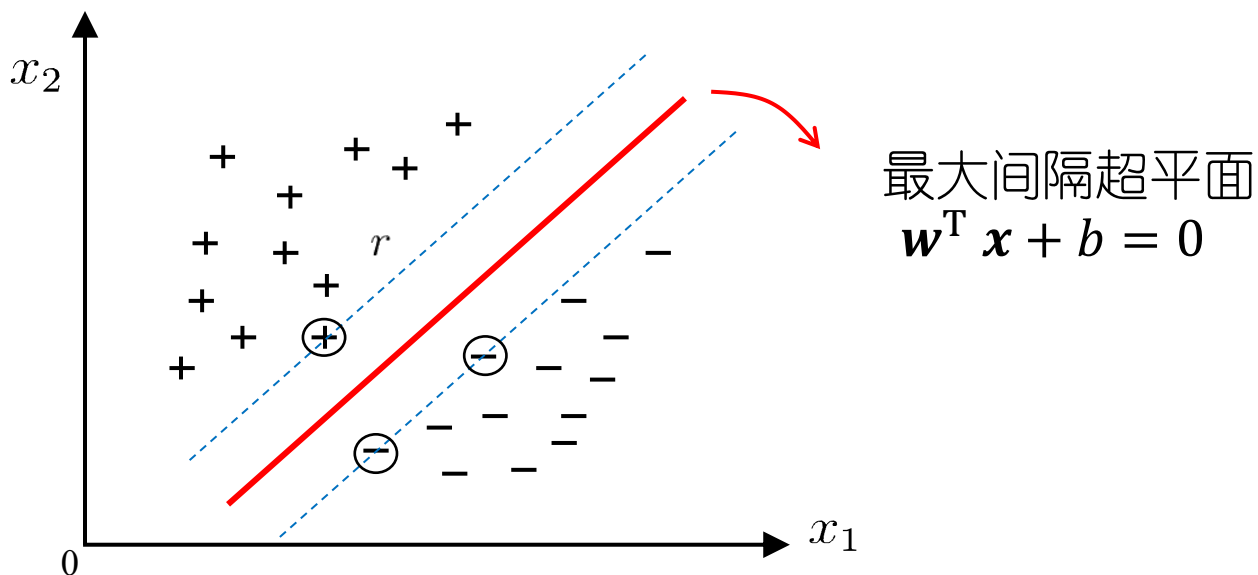


A: 直观意义上一般选取位于两类样本正中间的划分超平面, 泛化性能最好

最大间隔超平面

最佳平面：以最大间隔把两类样本分开的超平面

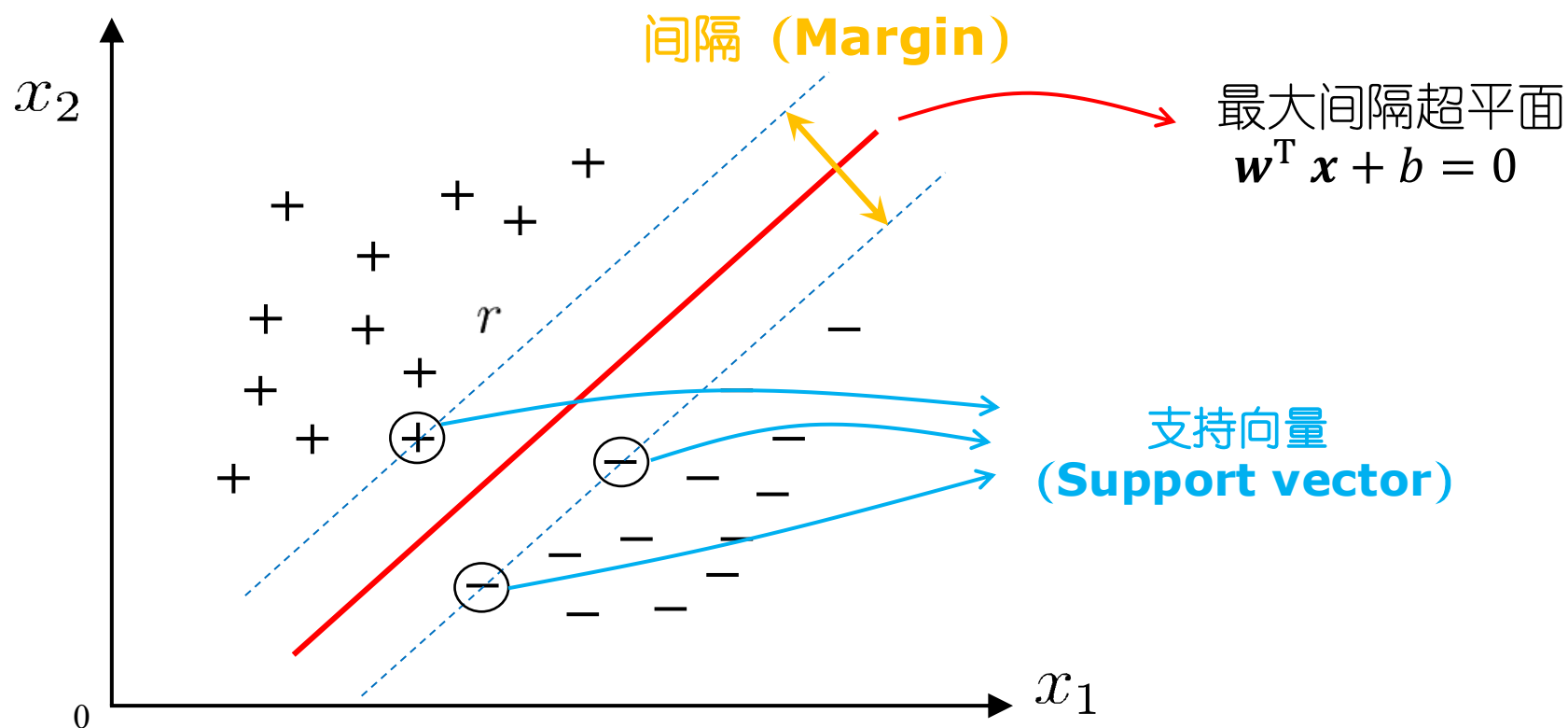
- 两类样本分别分割在该超平面的两侧；
- 两侧距离超平面最近的样本点到超平面的距离被最大化了。



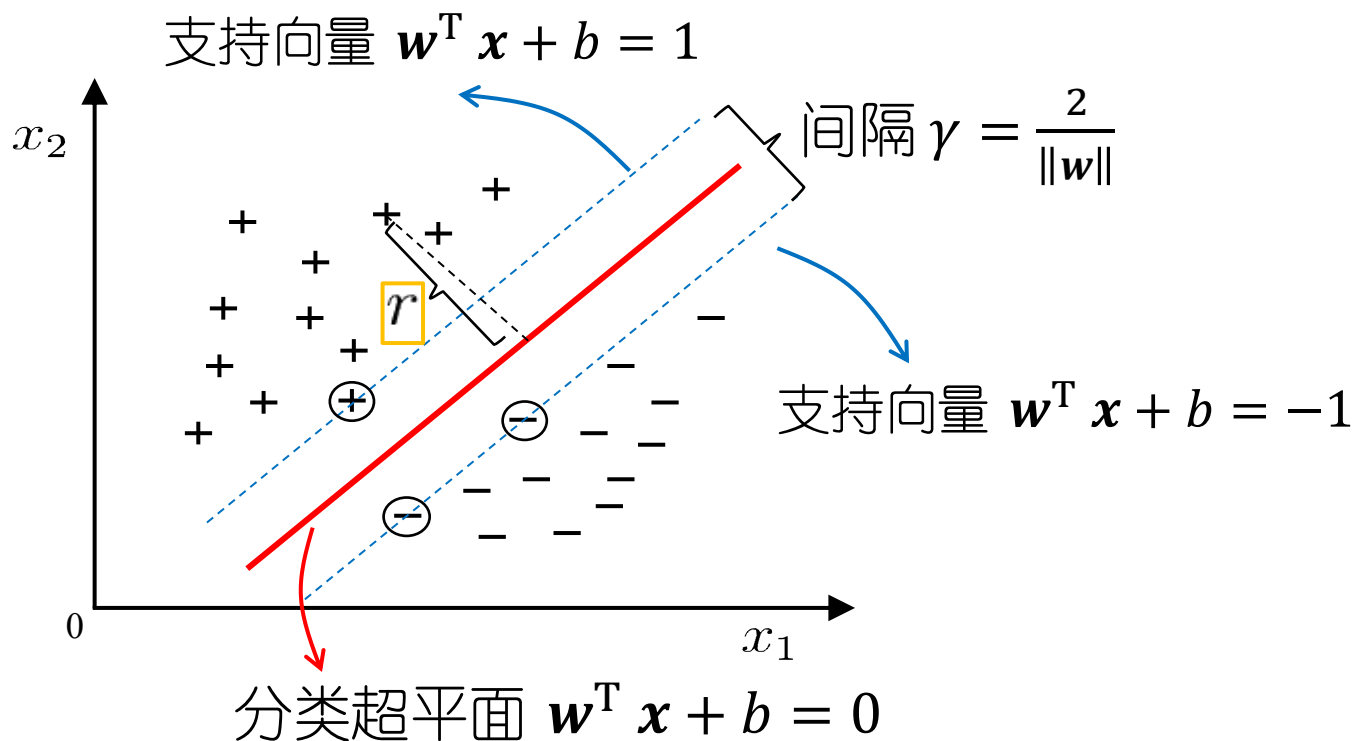
支持向量机：间隔与支持向量

支持向量机 (**Support Vector Machine, SVM**) :

最佳平面：以**最大间隔**把两类样本分开的超平面



支持向量机：间隔与支持向量

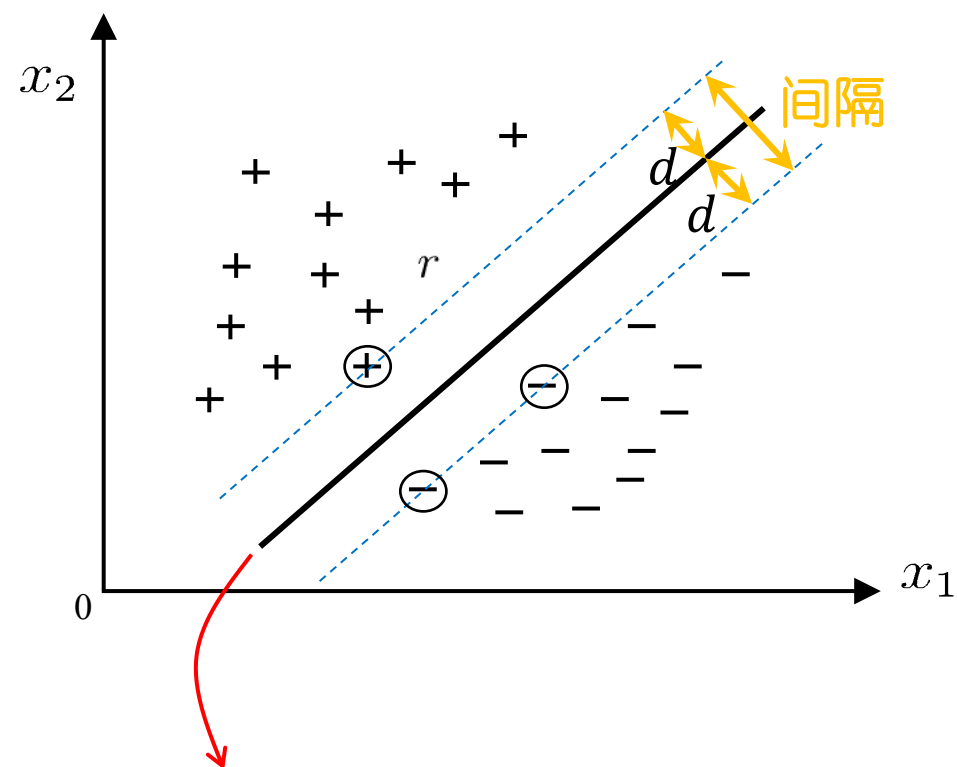


➤ 根据距离公式，空间中任意一点 $\mathbf{x}$ 到由 $(\mathbf{w}, b)$ 确定的超平面的距离为：

$$r = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$$

$$\text{其中, } \|\mathbf{w}\| = \sqrt{\omega_1^2 + \cdots + \omega_n^2}$$

SVM最优化问题



最大间隔超平面
 $w^T x + b = 0$

$$\begin{cases} \frac{w^T x + b}{\|w\|} \geq d & y = 1 \\ \frac{w^T x + b}{\|w\|} \leq -d & y = -1 \end{cases}$$

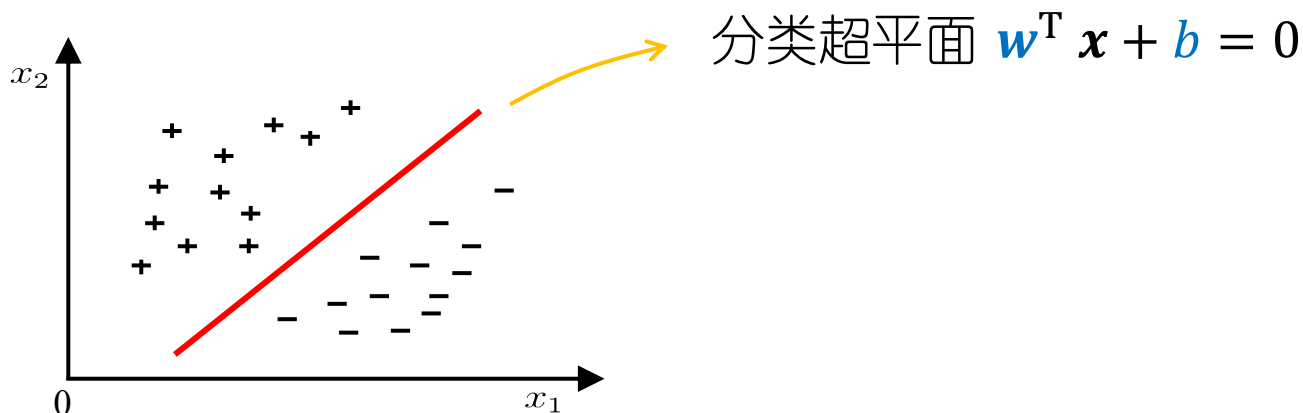


$$\begin{cases} \frac{w^T x + b}{\|w\|d} \geq 1 & y = 1 \\ \frac{w^T x + b}{\|w\|d} \leq -1 & y = -1 \end{cases}$$



$$\begin{cases} w^T x + b \geq 1 & y = 1 \\ w^T x + b \leq -1 & y = -1 \end{cases}$$

SVM最优化问题



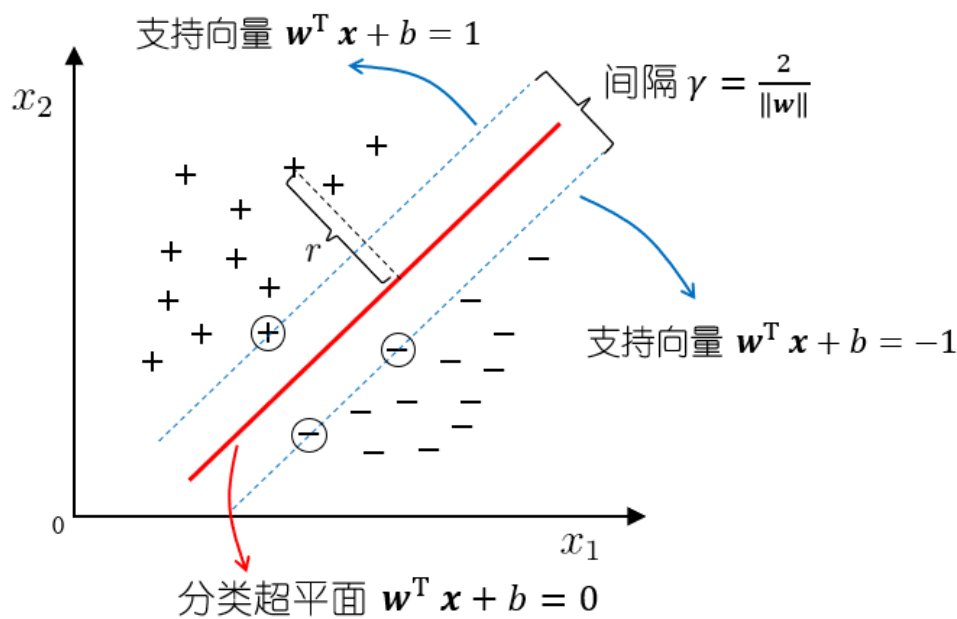
- 从直观意义上，希望所有样本距离分类超平面越远越好
- 根据距离公式，空间中任意一点 $\mathbf{x}$ 到由 $(\mathbf{w}, b)$ 确定的超平面的距离为：

$$r = \frac{|\mathbf{w}^T \mathbf{x} + b|}{\|\mathbf{w}\|}$$

- 由于参数 $(\mathbf{w}, b)$ 可以等比例变换，不加限制条件下距离 r 也可以等比例放缩

SVM最优化问题

- 定义两类样本中距离超平面最近的点到分类超平面的距离为 $\frac{1}{\|w\|}$ ，则经过最近点，且平行于分类超平面可支撑起两类样本间的间隔，距离为 $\frac{2}{\|w\|}$
- 经过最近点，且平行于分类超平面为支持向量，定义为 $w^T x + b = \pm 1$



SVM最优化问题

□ 支持向量机的基本型

- 目标：寻找到合适的参数 $\mathbf{w}, b$ ，使得两类样本之间的间隔距离 γ 最大

$$\begin{aligned} \arg \max_{\mathbf{w}, b} \quad & \frac{2}{\|\mathbf{w}\|} \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$



$$\begin{aligned} \arg \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$

支持向量机基本型

□ 支持向量机基本型

- 目标：寻找到合适的参数 $\mathbf{w}, b$ ，使得两类样本之间的间隔距离 γ 最大

$$\begin{aligned} \arg \max_{\mathbf{w}, b} \quad & \frac{2}{\|\mathbf{w}\|} \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$



$$\begin{aligned} \arg \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) > 1 \quad i = 1, 2, \dots, m. \end{aligned}$$

如何求解该优化问题 ? ? ?

大纲

- 支持向量机基本型
- 对偶问题
- SMO算法（选讲）
- 核函数
- 软间隔与正则化
- 支持向量回归

二分类任务 - 线性判别分析

□ 广义瑞利商 (generalized Rayleigh quotient)

$$J = \frac{\mathbf{w}^T \mathbf{S}_b \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}}$$

□ 令 $\mathbf{w}^T \mathbf{S}_w \mathbf{w} = 1$, 最大化广义瑞利商等价形式为

$$\begin{aligned} \min_{\mathbf{w}} \quad & -\mathbf{w}^T \mathbf{S}_b \mathbf{w} \\ \text{s.t.} \quad & \mathbf{w}^T \mathbf{S}_w \mathbf{w} = 1 \end{aligned}$$

□ 运用拉格朗日乘子法

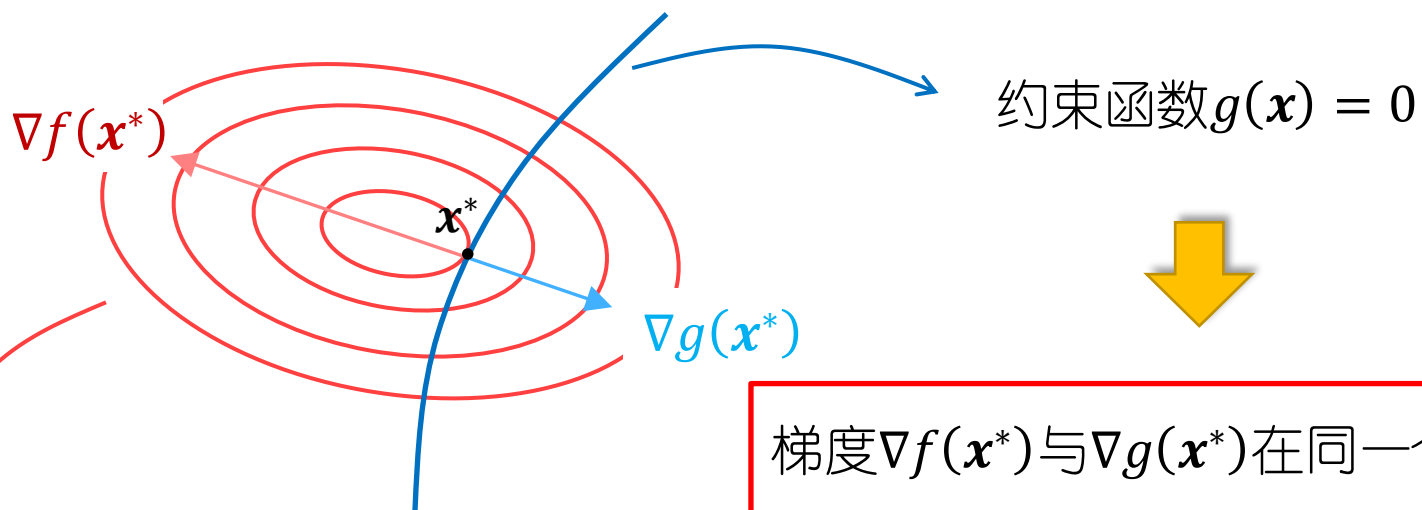
$$\mathbf{S}_b \mathbf{w} = \lambda \mathbf{S}_w \mathbf{w}$$

二分类任务

● 等式优化问题

$$\begin{aligned} \min_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{sub. to} \quad & g(\mathbf{x}) = 0 \end{aligned}$$

- 对于约束曲面 $g(\mathbf{x}) = 0$ 上的任意点 $\mathbf{x}$ ，该点的梯度 $\nabla g(\mathbf{x}^*)$ 正交于约束曲面
- 在最优 $\mathbf{x}^*$ ，目标函数在该点的梯度 $\nabla f(\mathbf{x}^*)$ 正交于约束曲面



函数 $f(\mathbf{x})$ 等高线，越往里数值越小

梯度 $\nabla f(\mathbf{x}^*)$ 与 $\nabla g(\mathbf{x}^*)$ 在同一个方向上

$$\nabla f(\mathbf{x}^*) + \lambda \nabla g(\mathbf{x}^*) = \mathbf{0}$$

二分类任务

- 优化问题

$$\begin{aligned} \min_{\mathbf{w}} \quad & -\mathbf{w}^T \mathbf{S}_b \mathbf{w} \quad \xrightarrow{\quad} f(\mathbf{w}) \\ \text{s.t.} \quad & \mathbf{w}^T \mathbf{S}_w \mathbf{w} = 1 \quad \xrightarrow{\quad} g(\mathbf{w}) = 0 \end{aligned}$$

$$\begin{aligned} \nabla f(\mathbf{w}) &= -\mathbf{S}_b \mathbf{w} - \mathbf{S}_b^T \mathbf{w} \\ &= -2\mathbf{S}_b \mathbf{w} \end{aligned}$$

$$\begin{aligned} \nabla g(\mathbf{w}) &= \mathbf{S}_w \mathbf{w} + \mathbf{S}_w^T \mathbf{w} \\ &= 2\mathbf{S}_w \mathbf{w} \end{aligned}$$

梯度 $\nabla f(\mathbf{w}^*)$ 与 $\nabla g(\mathbf{w}^*)$ 在同一个方向上

$$\nabla f(\mathbf{w}^*) + \lambda \nabla g(\mathbf{w}^*) = \mathbf{0}$$

$$\mathbf{S}_b \mathbf{w}^* = \lambda \mathbf{S}_w \mathbf{w}^*$$

$$\mathbf{S}_b \mathbf{w} = (\mu_0 - \mu_1) \boxed{(\mu_0 - \mu_1)^T \mathbf{w}} \quad \text{标量}$$

$$\text{不妨令 } \mathbf{S}_b \mathbf{w}^* = \lambda (\mu_0 - \mu_1)$$

$$\mathbf{w}^* = \mathbf{S}_w^{-1} (\mu_0 - \mu_1)$$

可用奇异值分解方法求解

$\mathbf{S}_b$, $\mathbf{S}_w$ 是对称矩阵

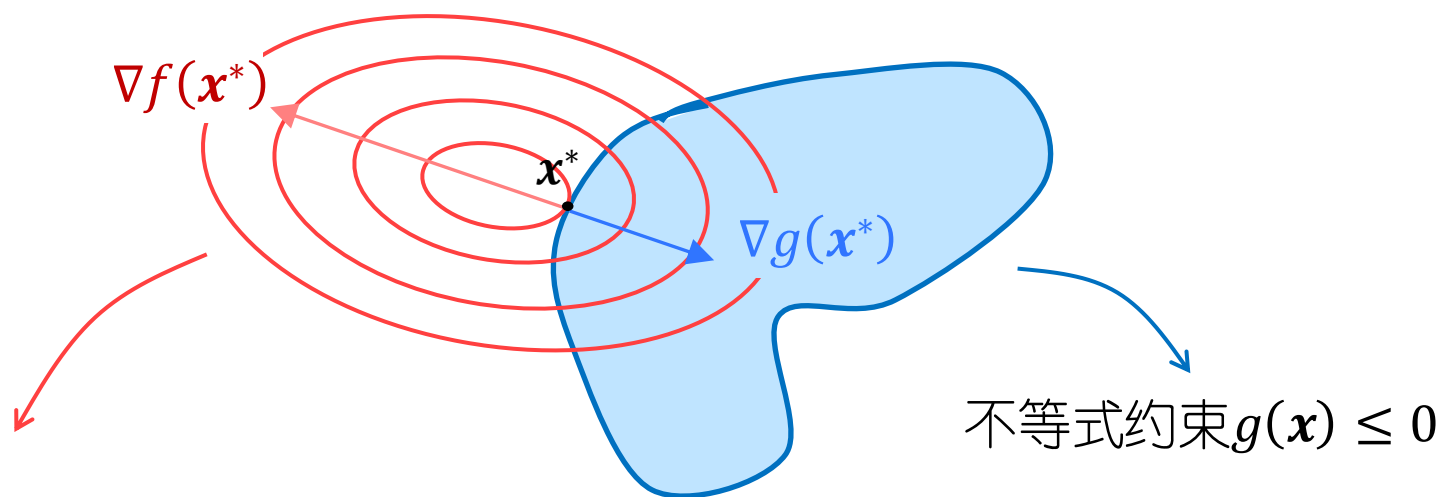
对偶问题

□ 拉格朗日乘子法

将有约束的最优化问题转化为无约束优化问题的方法

不等式约束优化问题

$$\begin{array}{ll}\min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{sub. to} & g(\mathbf{x}) \leq 0\end{array}$$



目标函数 $f(\mathbf{x})$ 等高线，越往里数值越小

不等式约束 $g(\mathbf{x}) \leq 0$

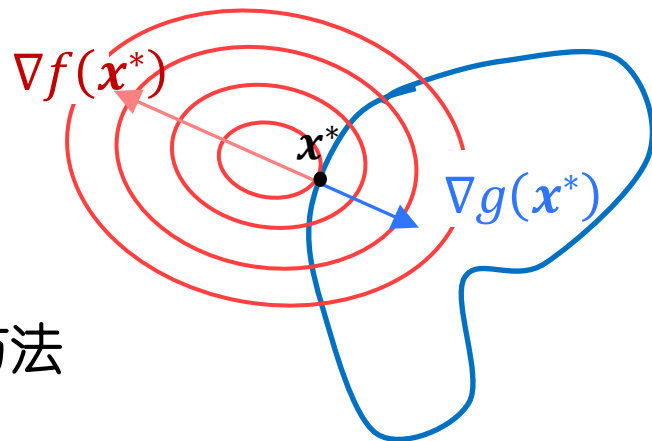
拉格朗日乘子法

□ 等式约束优化问题

将有约束的最优化问题转化为无约束优化问题的方法

➤ 等式约束（即最优解在约束的边界上）

- 最优解发生在约束边界与目标函数等高线相切的位置
- 目标函数等高线与最优解梯度 $\nabla f(\mathbf{x}^*)$ 垂直

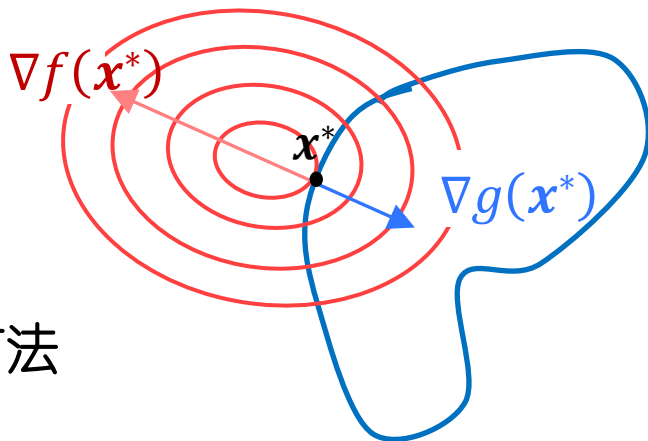


梯度 $\nabla f(\mathbf{x}^*)$ 与 $\nabla g(\mathbf{x}^*)$ 在同一个方向上, $\nabla f(\mathbf{x}^*) + \lambda \nabla g(\mathbf{x}^*) = \mathbf{0}$

拉格朗日乘子法

□ 等式约束优化问题

将有约束的最优化问题转化为无约束优化问题的方法



$$\begin{array}{ll} \min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{sub. to} & g(\mathbf{x}) = 0 \end{array}$$



引入拉格朗日乘子 λ

拉格朗日函数：

$$L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$$



求等式约束优化问题，需满足 $\frac{\partial L}{\partial \mathbf{x}} = \nabla f(\mathbf{x}^*) + \lambda \nabla g(\mathbf{x}^*) = 0$, $\frac{\partial L}{\partial \lambda} = g(\mathbf{x}^*) = 0$

拉格朗日乘子法

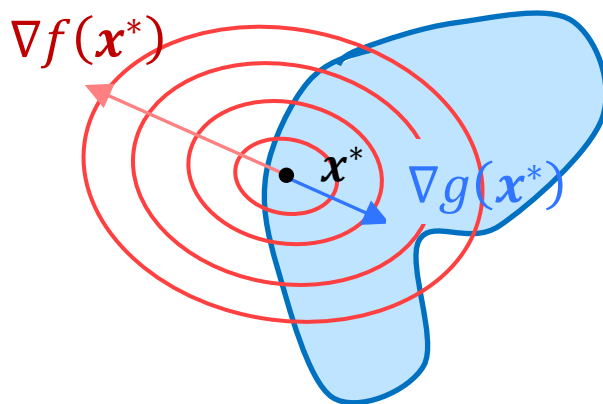
□ 不等式约束优化问题

将有约束的最优化问题转化为无约束优化问题的方法

不等式约束优化问题

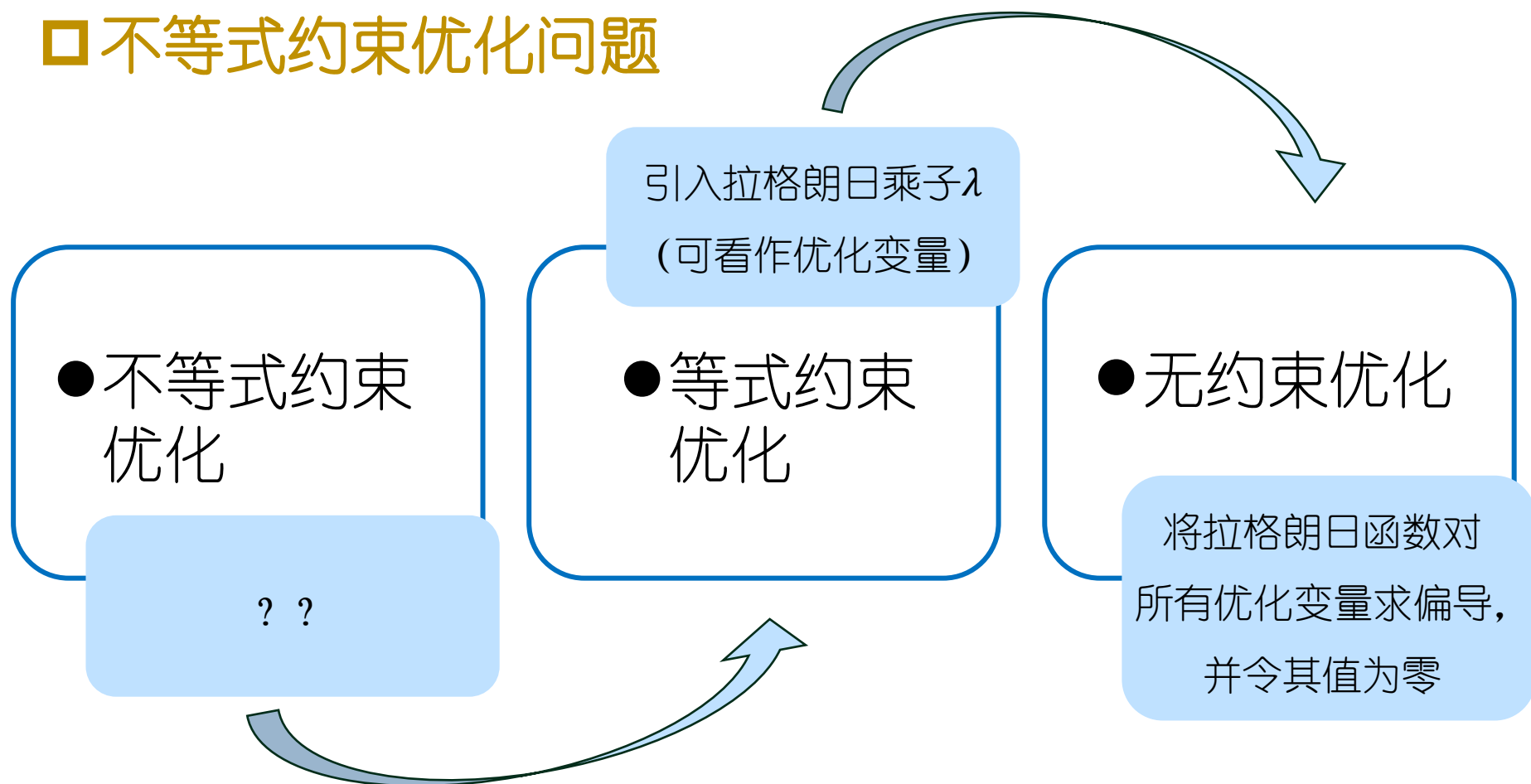
$$\begin{array}{ll} \min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{sub. to} & g(\mathbf{x}) \leq 0 \end{array}$$

➤ 由于约束条件 $g(\mathbf{x}) \leq 0$ 的存在，最优解不能用 $\nabla f(\mathbf{x}^*) = \mathbf{0}$ 的方式直接求解



拉格朗日乘子法

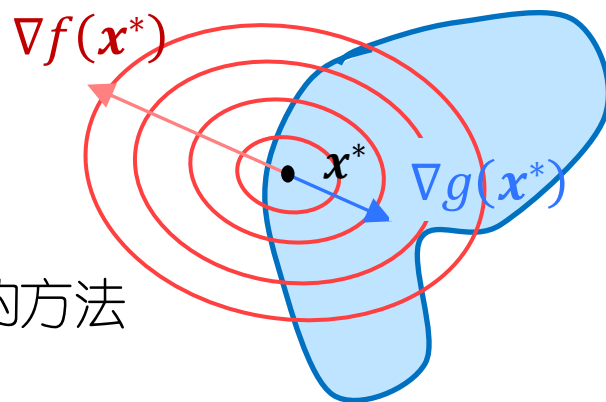
□ 不等式约束优化问题



拉格朗日乘子法

□ 不等式约束优化问题

将有约束的最优化问题转化为无约束优化问题的方法



不等式约束优化问题

$$\begin{array}{ll} \min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{sub. to} & g(\mathbf{x}) \leq 0 \end{array}$$

- 由于约束条件 $g(\mathbf{x}) \leq 0$ 的存在，最优解不能用 $\nabla f(\mathbf{x}^*) = \mathbf{0}$ 的方式直接求解
- 不等式约束 (1. 最优解在约束的边界内部 $g(\mathbf{x}^*) < 0$)

此时约束 $g(\mathbf{x}) \leq 0$ 将不起作用，最优解是在 $\lambda = 0$ 的情况下，对拉格朗日函数 $L(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda g(\mathbf{x})$ 关于 $\mathbf{x}$ 求导置零的结果，即

$$\frac{\partial L}{\partial \mathbf{x}} = \nabla f(\mathbf{x}^*) = \mathbf{0}$$



$$g(\mathbf{x}^*) < 0 \text{ 时, } \lambda = 0$$

拉格朗日乘子法

□ 不等式约束优化问题

将有约束的最优化问题转化为无约束优化问题的方法

不等式约束优化问题

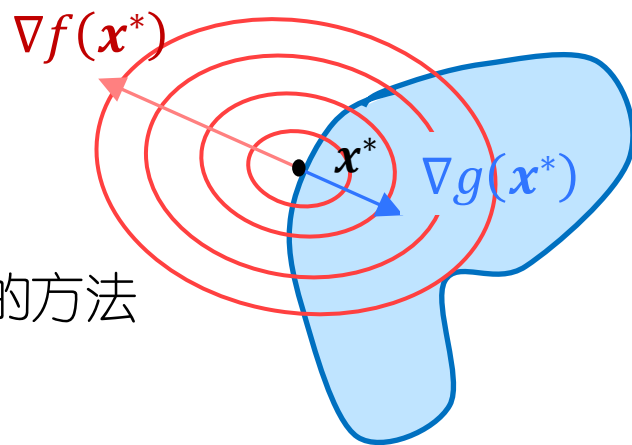
$$\begin{array}{ll}\min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{sub. to} & g(\mathbf{x}) \leq 0\end{array}$$

- 由于约束条件 $g(\mathbf{x}) \leq 0$ 的存在，最优解不能用 $\nabla f(\mathbf{x}^*) = \mathbf{0}$ 的方式直接求解
- 不等式约束（2. 最优解在约束的边界 $g(\mathbf{x}^*) = 0$ 上）

同等式约束情况类似，但最优解在目标函数梯度 $\nabla f(\mathbf{x}^*)$ 与约束函数梯度 $\nabla g(\mathbf{x}^*)$ 方向相反，为满足 $\nabla f(\mathbf{x}^*) + \lambda \nabla g(\mathbf{x}^*) = \mathbf{0}$ ， $\lambda > 0$



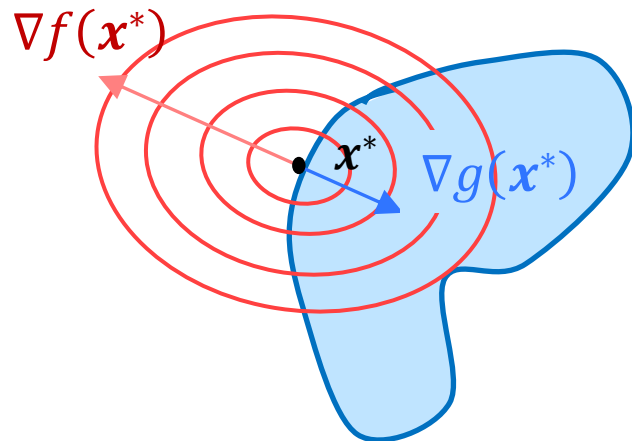
$$g(\mathbf{x}^*) = 0 \text{ 时, } \lambda > 0$$



拉格朗日乘子法

□ 不等式约束优化问题

- 约束不起作用时: $g(x^*) < 0$, $\lambda = 0$
- 约束条件起作用时: $g(x^*) = 0$, $\lambda > 0$



➤ 不等式约束

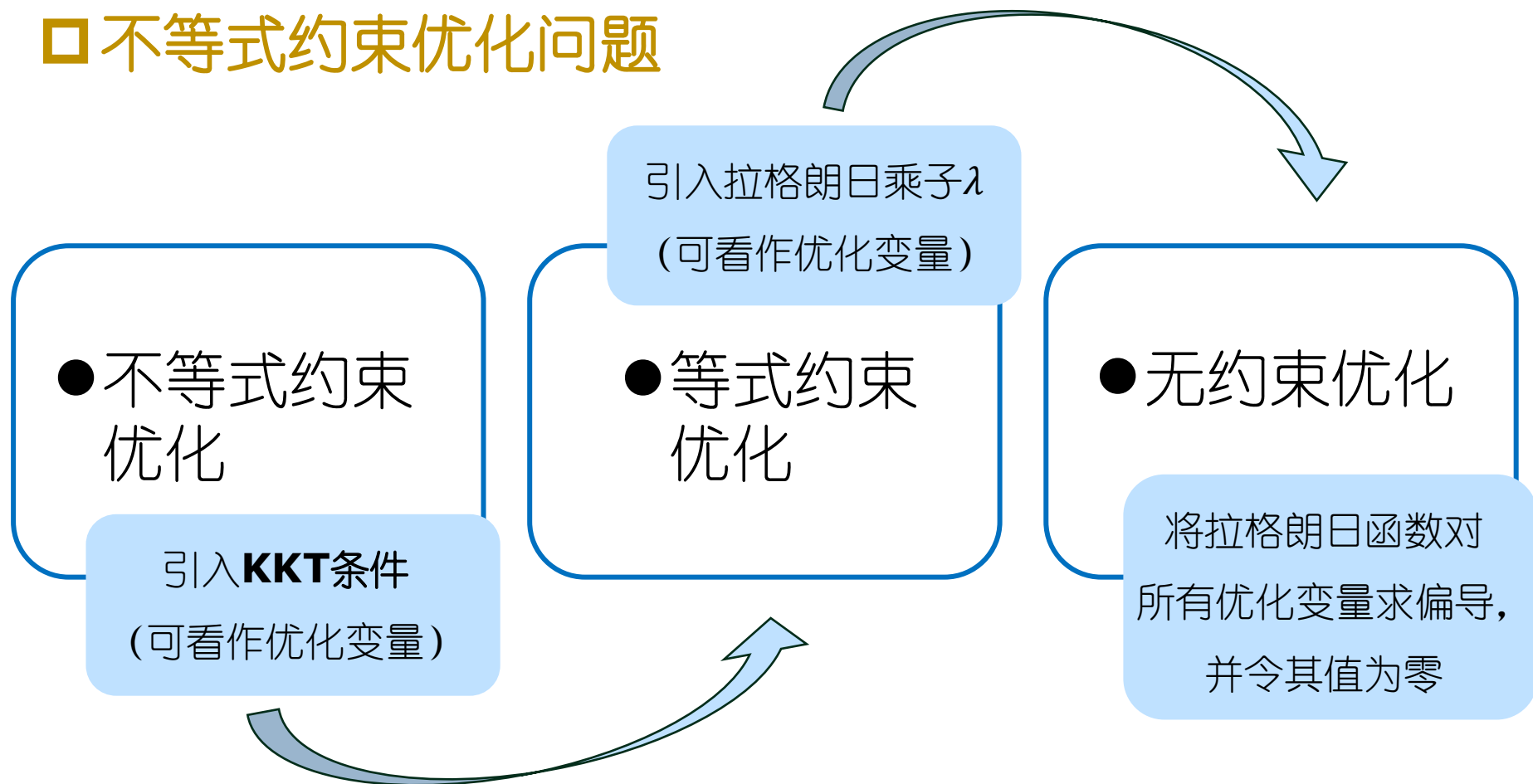
综合不等式约束2种情况, 在不等式约束条件下, 最优解 x^* 是在满足条件

$$\text{KKT条件: } \begin{cases} g(x) \leq 0 \\ \lambda \geq 0 \\ \lambda g(x) = 0 \end{cases}$$

下对拉格朗日函数求导置零的结果

拉格朗日乘子法

□ 不等式约束优化问题



对偶问题

□ 支持向量机基本型

- 目标：寻找到合适的参数 $\mathbf{w}, b$ ，使得两类样本之间的间隔距离 γ 最大

$$\arg \min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2$$

$$\text{s.t. } y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m.$$



$$\begin{array}{ll} \min_{\mathbf{x}} & f(\mathbf{x}) \\ \text{sub.to} & g(\mathbf{x}) \leq 0 \end{array}$$

$$\begin{aligned} f(\boldsymbol{\omega}) &= \frac{1}{2} \|\boldsymbol{\omega}\|^2 \\ g(\boldsymbol{\omega}) &= 1 - y_i(\boldsymbol{\omega}^\top \mathbf{x}_i + b) \end{aligned}$$

对偶问题

➤ 不等式约束

综合不等式约束2种情况，在不等式约束条件下，最优解 $\mathbf{x}^*$ 是在满足条件

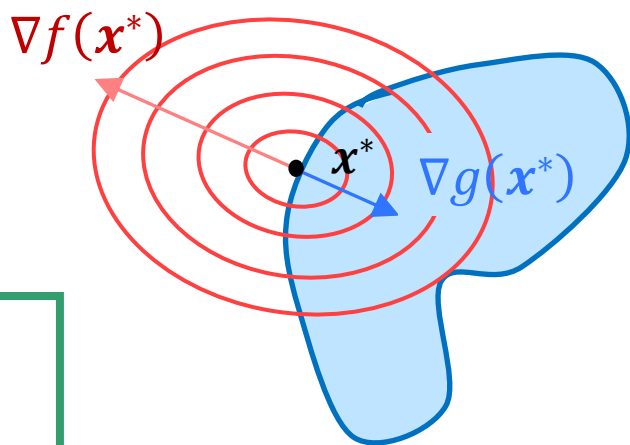
$$\text{KKT条件: } \begin{cases} g(\mathbf{x}) \leq 0 \\ \lambda \geq 0 \\ \lambda g(\mathbf{x}) = 0 \end{cases}$$

下对拉格朗日函数求导置零的结果



$$\begin{aligned} f(\omega) &= \frac{1}{2} \|\omega\|^2 \\ g(\omega) &= 1 - y_i(\omega^T x_i + b) \end{aligned}$$

$$L(w, b, \lambda) = \frac{1}{2} \|w\|^2 + \sum_{i=1}^n \lambda_i [1 - y_i(w^T x_i + b)]$$



对偶问题: KKT条件

$$\begin{cases} g(\mathbf{x}) \leq 0 \\ \lambda \geq 0 \\ \lambda g(\mathbf{x}) = 0 \end{cases} \longrightarrow \begin{cases} g_i(\mathbf{x}) \leq 0 \\ \alpha_i \geq 0 \\ \alpha_i g_i(\mathbf{x}) = 0 \end{cases}$$

$$\downarrow g(\omega) = 1 - f(x)y_i$$

$$\begin{cases} f(x)y_i - 1 \geq 0 \\ \alpha_i \geq 0 \\ \alpha_i(f(x)y_i - 1) = 0 \end{cases}$$

对偶问题

- 用拉格朗日乘子法求解支持向量机

- Step. 1 引入拉格朗日乘子 $\alpha_i \geq 0$ 得到拉格朗日函数

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i (\mathbf{w}^T \mathbf{x}_i + b))$$

- Step. 2 令 $L(\mathbf{w}, b, \boldsymbol{\alpha})$ 对 $\mathbf{w}$ 和 b 偏导置零，得

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i, \quad \sum_{i=1}^m \alpha_i y_i = 0.$$

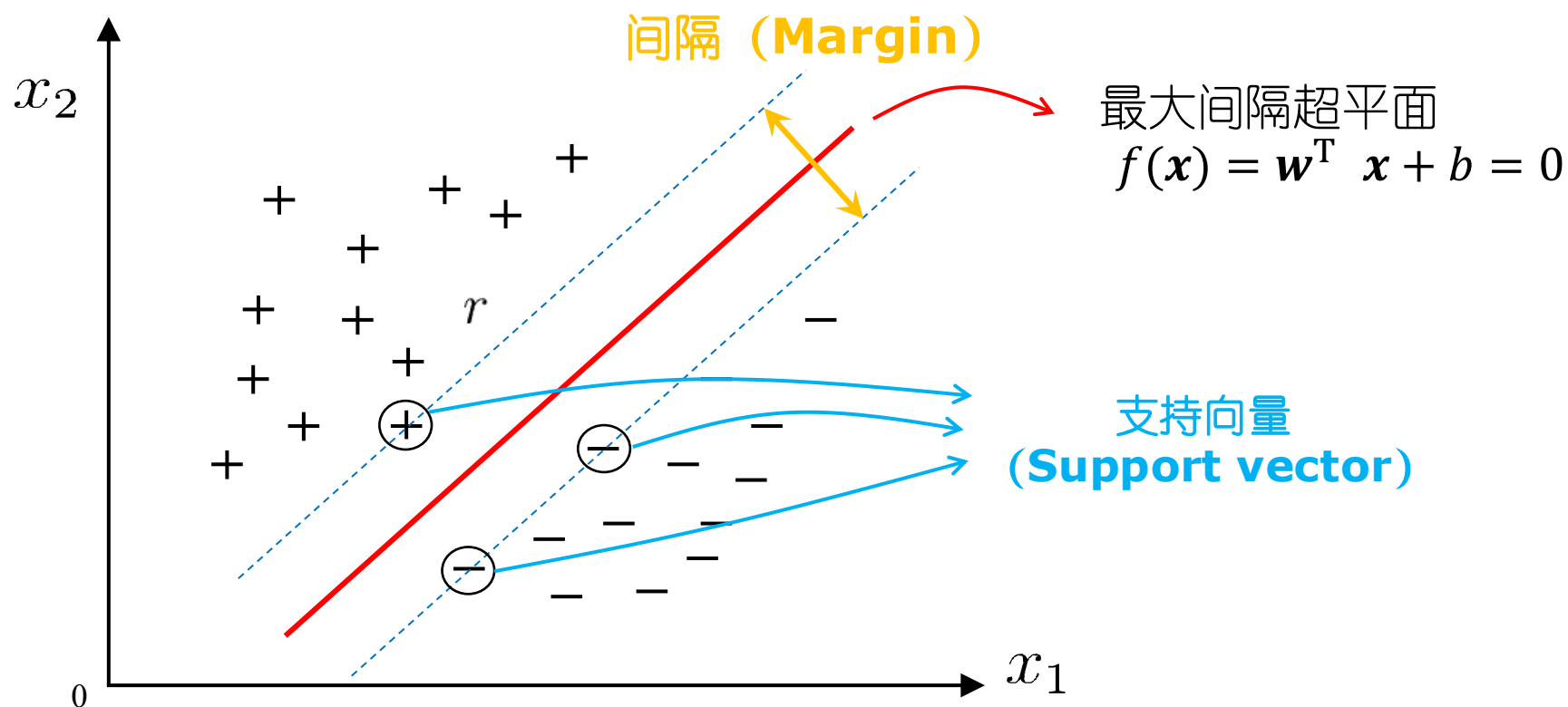
- Step. 3 将上式代回拉格朗日函数，并在拉格朗日乘子下最大化拉格朗日函数，得对偶问题：

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

支持向量机：间隔与支持向量

支持向量机 (**Support Vector Machine, SVM**) :

最佳平面：以**最大间隔**把两类样本分开的超平面



SVM最优化问题

□ 支持向量机的基本型

- 目标：寻找到合适的参数 $\mathbf{w}, b$ ，使得两类样本之间的间隔距离 γ 最大

$$\begin{aligned} \arg \max_{\mathbf{w}, b} \quad & \frac{2}{\|\mathbf{w}\|} \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$



$$\begin{aligned} \arg \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$

对偶问题

KKT条件:

$$f(x)y_i - 1 \geq 0$$

$$\alpha_i \geq 0$$

$$\alpha_i(f(x)y_i - 1) = 0$$

- 用拉格朗日乘子法求解支持向量机

- Step. 1 引入拉格朗日乘子 $\alpha_i \geq 0$ 得到拉格朗日函数

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 + \sum_{i=1}^m \alpha_i (1 - y_i (\mathbf{w}^T \mathbf{x}_i + b))$$

- Step. 2 令 $L(\mathbf{w}, b, \boldsymbol{\alpha})$ 对 $\mathbf{w}$ 和 b 偏导置零, 得

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i, \quad \sum_{i=1}^m \alpha_i y_i = 0.$$

- Step. 3 将上式代回拉格朗日函数, 并在拉格朗日乘子下最大化拉格朗日函数, 得对偶问题:

$$\begin{aligned} \min_{\boldsymbol{\alpha}} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

对偶问题

□ 支持向量机基本型

- 目标：寻找到合适的参数 $\mathbf{w}, b$ ，使得两类样本之间的间隔距离 γ 最大

$$\begin{aligned} \arg \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$



$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

稀疏性

- 求解对偶问题 α^* ，并对于任意 $\alpha_j^* > 0$ ，可得样本空间的分类超平面

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$

$$\mathbf{w}^* = \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i$$
$$b^* = y_j - \sum_{i=1}^m \alpha_i^* y_i (\mathbf{x}_i \cdot \mathbf{x}_j)$$

- 支持向量机完成分类

$$\text{当 } f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b \geq +1$$

$$\text{当 } f(\mathbf{x}_i) = \mathbf{w}^T \mathbf{x}_i + b \leq -1$$



$$y_i = +1$$

$$y_i = -1$$

稀疏性

- 求解对偶问题 α^* ，并对于任意 $\alpha_j^* > 0$ ，可得样本空间的分类超平面

$$f(\mathbf{x}) = \mathbf{w}^T \mathbf{x} + b$$
$$\mathbf{w}^* = \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i$$
$$b^* = y_j - \sum_{i=1}^m \alpha_i^* y_i (\mathbf{x}_i \cdot \mathbf{x}_j)$$


- 注意观察 $\mathbf{w}^*$ 由所有样本数据共同构成，根据KKT条件

$$\begin{cases} \alpha_i \geq 0, \\ y_i f(\mathbf{x}_i) \geq 1, \end{cases}$$

$$\alpha_i (y_i f(\mathbf{x}_i) - 1) = 0.$$

对于任意样本 $(\mathbf{x}_i, y_i)$ ，有

- $\alpha_i = 0 \rightarrow$ 不会对模型有影响
- $y_i f(\mathbf{x}_i) = 1 \rightarrow$ 样本在支持向量上

稀疏性：训练完成后，大部分的训练样本都无需保留，
SVM最终模型仅与支持向量有关

大纲

- 支持向量机基本型
- 对偶问题
- **SMO**算法（选讲）
- 核函数
- 软间隔与正则化
- 支持向量回归

对偶问题

对偶问题公式：

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^{\top} \mathbf{x}_j - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

- 仍然是一个二次规划问题，可使用通用的二次规划算法来求解；
- 计算规模正比于训练样本数，在实际任务中造成很大的计算开销。

如何高效地计算出 α_i ？ ？

求解方法 - SMO

□ 常规求解优化问题方法

- 最优解情况

所有拉格朗日乘子变量 α_i 满足KKT条件

- 非最优解情况

有拉格朗日乘子变量 α_i 不能满足KKT条件

□ 更新思想

- 寻找到不满足KKT条件的拉格朗日乘子 α_i 并对其进行更新，使更新后的目标函数能进一步缩小

求解方法 - SMO

- 基本思路：不断执行如下两个步骤直至收敛。
 - 第一步：选取一对需更新的变量 α_i 和 α_j 。
 - 第二步：固定 α_i 和 α_j 以外的参数，求解对偶问题更新 α_i 和 α_j 。

- 仅考虑 α_i 和 α_j 时，对偶问题的约束变为

$$\alpha_i y_i + \alpha_j y_j = - \sum_{k \neq i, j} \alpha_k y_k, \quad \alpha_i \geq 0, \quad \alpha_j \geq 0.$$

用一个变量表示另一个变量，回代入对偶问题可得一个单变量的二次规划，该问题具有闭式解。

- 偏移项 b ：通过支持向量来确定。

求解方法 - SMO

□ 最小序列方法(SMO)基本思想

每次固定其它维度，只对单一变量 α_i 进行更新

$$\max_{\alpha} \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j)$$


$$s. t. \sum_{i=1}^m \alpha_i y_i = 0,$$

$$0 \leq \alpha_i \leq C, i = 1, 2, \dots, m$$

➤ 但由于约束条件的限制，每次至少要更新两个变量，假设更新变量为 α_1, α_2

求解方法 - SMO

□ 最小序列方法(SMO)基本思想

选择两个变量，其它变量固定

SMO将对偶问题转化成一系列子问题

$$\begin{aligned} \min_{\alpha_1, \alpha_2} \quad & W(\alpha_1, \alpha_2) = \frac{1}{2}K_{11}\alpha_1^2 + \frac{1}{2}K_{22}\alpha_2^2 + y_1y_2K_{12}\alpha_1\alpha_2 \\ & -(\alpha_1 + \alpha_2) + y_1\alpha_1 \sum_{i=3}^N y_i\alpha_i K_{i1} + y_2\alpha_2 \sum_{i=3}^N y_i\alpha_i K_{i2} \\ \text{s.t.} \quad & \alpha_1y_1 + \alpha_2y_2 = -\sum_{i=3}^N y_i\alpha_i = \zeta \\ & 0 \leq \alpha_i \leq C, \quad i = 1, 2 \end{aligned}$$

此时由于其它变量 α_i 固定， ζ 是确定的，可以根据约束条件直接求得 α_1 与 α_2 之间的关系，代回原问题，在**没有 $0 \leq \alpha_i \leq C$ 的约束条件**下可以直接通过对目标 $W(\alpha_1, \alpha_2)$ 偏导置零求得目标函数极值点

求解方法 - SMO

□ 最小序列方法(SMO)基本思想

- 假设初始解为 α_1^{old} , α_2^{old} , 最优解为 α_1^{new} , α_2^{new}
- 由于约束条件, 最优解 α_2^{new} 必然满足区间要求

$$L \leq \alpha_2^{new} \leq H$$

- 由于 $\alpha_1 y_1 + \alpha_2 y_2 = \zeta$, 分析可得

- 当 $y_1 \neq y_2$ 时

$$L = \max(0, \alpha_2^{old} - \alpha_1^{old}) \quad H = \min(C, C + \alpha_2^{old} - \alpha_1^{old})$$

- 当 $y_1 = y_2$ 时

$$L = \max(0, \alpha_2^{old} + \alpha_1^{old} - C) \quad H = \min(C, \alpha_2^{old} + \alpha_1^{old})$$

求解方法 - SMO

□ 最小序列方法(SMO)基本思想

- 为叙述简单，定义样本输出为

$$g(\mathbf{x}) = \sum_i^m \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b$$

- 输出与真实标签之间的差异为

$$E_i = g(\mathbf{x}_i) - y_i$$

求解方法 - SMO

□ 最小序列方法(SMO)基本思想

- 不考虑约束条件计算出的最小化 $W(\alpha_1, \alpha_2)$ 解为

$$\alpha_2^{new,unc} = \alpha_2^{old} + \frac{y_2(E_1 - E_2)}{\eta}$$

- 其中

$$\eta = K_{11} + K_{22} - 2K_{12}$$

- 考虑约束条件对 α_1^{new} , α_2^{new} 进行更新

$$\alpha_2^{new} = \begin{cases} H, & \alpha_2^{new,unc} > H \\ \alpha_2^{new,unc}, & L \leq \alpha_2^{new,unc} \leq H \\ L, & \alpha_2^{new,unc} < L \end{cases}$$

$$\alpha_1^{new} = \alpha_1^{old} + y_1 y_2 (\alpha_2^{old} - \alpha_2^{new})$$

求解方法 - SMO

□ 更新变量选择方法

- SMO算法在每个子问题中选择两个变量优化，其中至少一个是违反KKT条件的
- 1. 第一个变量选择：外循环
 - 违反KKT条件最严重的样本点
(依次检查每个样本点是否满足KKT条件)

$$\alpha_i = 0 \leftrightarrow y_i g(x_i) \geq 1$$

$$0 < \alpha_i < C \leftrightarrow y_i g(x_i) \leq 1$$

$$\alpha_i = C \leftrightarrow y_i g(x_i) \leq 1$$

$$g(x_i) = \sum_{j=1}^N \alpha_j y_j K(x_i, x_j) + b$$

求解方法 - SMO

□ 更新变量选择方法

- SMO算法在每个子问题中选择两个变量优化，其中至少一个是违反KKT条件的
- 2. 第二个变量选择：内循环
 - 希望使目标函数有足够大的变化
选择 α_2 ,使得对应的 $|E_1 - E_2|$ 最大

求解方法 - SMO

SMO算法

SMO算法

- 输入：训练数据集 $T = \{(x_1, y_1), (x_2, y_2), \dots, (x_N, y_N)\}$
 $x_i \in \mathcal{X} = R^n, y_i \in \mathcal{Y} = \{+1, -1\}, i = 1, 2, \dots, N$ ，精度 ε

- 输出：近似解 α

(1) 取初值 $\alpha^{(0)} = 0$ ，令 $k = 0$

(2) 选取优化变量 α_1^k, α_2^k ，解析求解两个变量的最优化问题，求得最优解 $\alpha_1^{(k+1)}, \alpha_2^{(k+1)}$ ，更新 α 为 $\alpha^{(k+1)}$ ；

(3) 若在精度 ε 范围内满足停机条件

$$\sum_{i=1}^N \alpha_i y_i = 0$$

$$0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, N$$

则转(4); 否则令 $k = k + 1$, 转(2);

(4) 取 $\hat{\alpha} = \alpha^{(k+1)}$

$$y_i \cdot g(x_i) = \begin{cases} \geq 1, & \{x_i | \alpha_i = 0\} \\ = 1, & \{x_i | 0 < \alpha_i < C\} \\ \leq 1, & \{x_i | \alpha_i = C\} \end{cases}$$

$$g(x_i) = \sum_{j=1}^N \alpha_j y_j K(x_j, x_i) + b$$

求解方法 - SMO

□ 一次求出最优解 α_i^* 之后，可求出：

$$\mathbf{w}^* = \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i$$

□ 偏移项 b ：通过支持向量来确定

对于支持向量，有 $y_s f(x_s) = 1$ ，即 $y_s(\mathbf{w}^* x_s + b) = 1$

两边同乘 y_s ，有 $y_s^2(\mathbf{w}^* x_s + b) = y_s$

由于 $y_s^2 = 1$ ，有 $b = y_s - \mathbf{w}^* x_s$

一般取所有支持向量求解的平均值，有：

$$b = \frac{1}{|S|} \sum_{s \in S} (y_s - \mathbf{w}^* x_s)$$

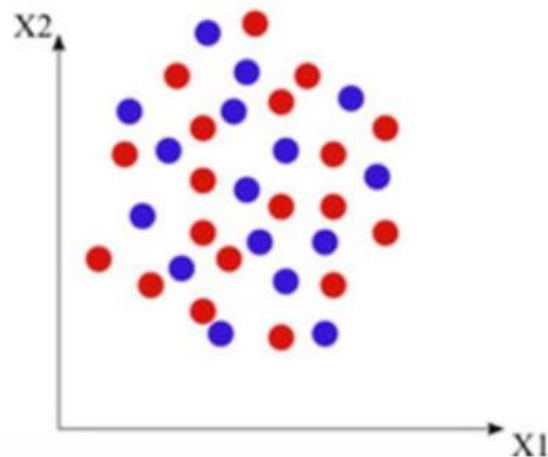
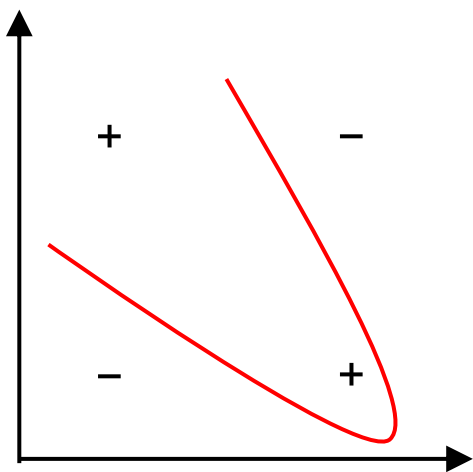
大纲

- 支持向量机基本型
- 对偶问题
- SMO算法（选讲）
- 核函数
- 软间隔与正则化
- 支持向量回归

线性不可分

□ 面临的问题

- 如果在原始空间正负样本线性不可分，应当如何如理？
- 直接处理非线性问题非常复杂，能否将非线性问题转化为线性问题？

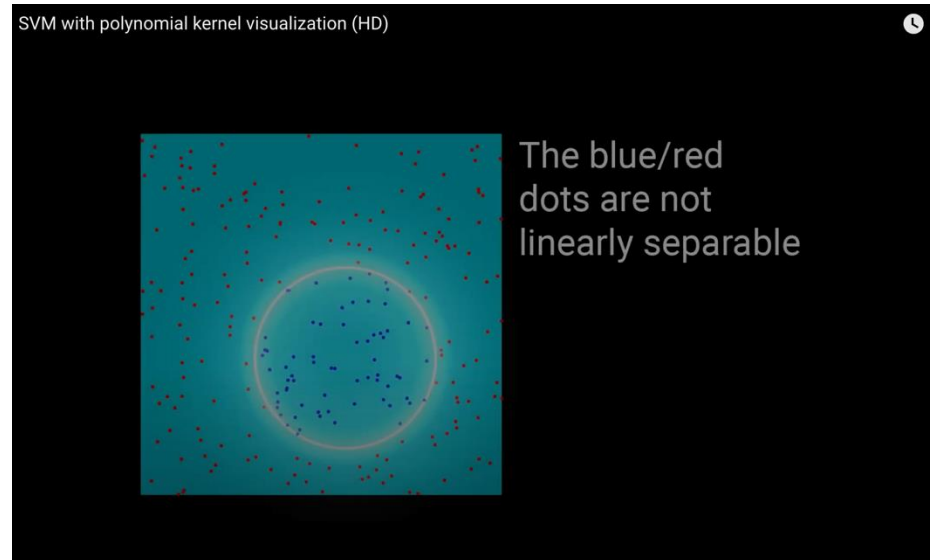
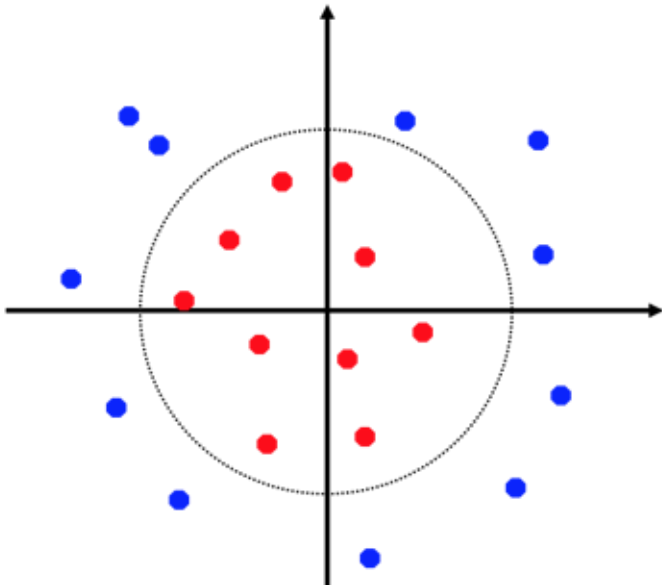


A: 将样本从原始空间映射到一个更高维的特征空间，使得样本在高维空间内线性可分

线性不可分

当线性模型无法有效划分样本时，

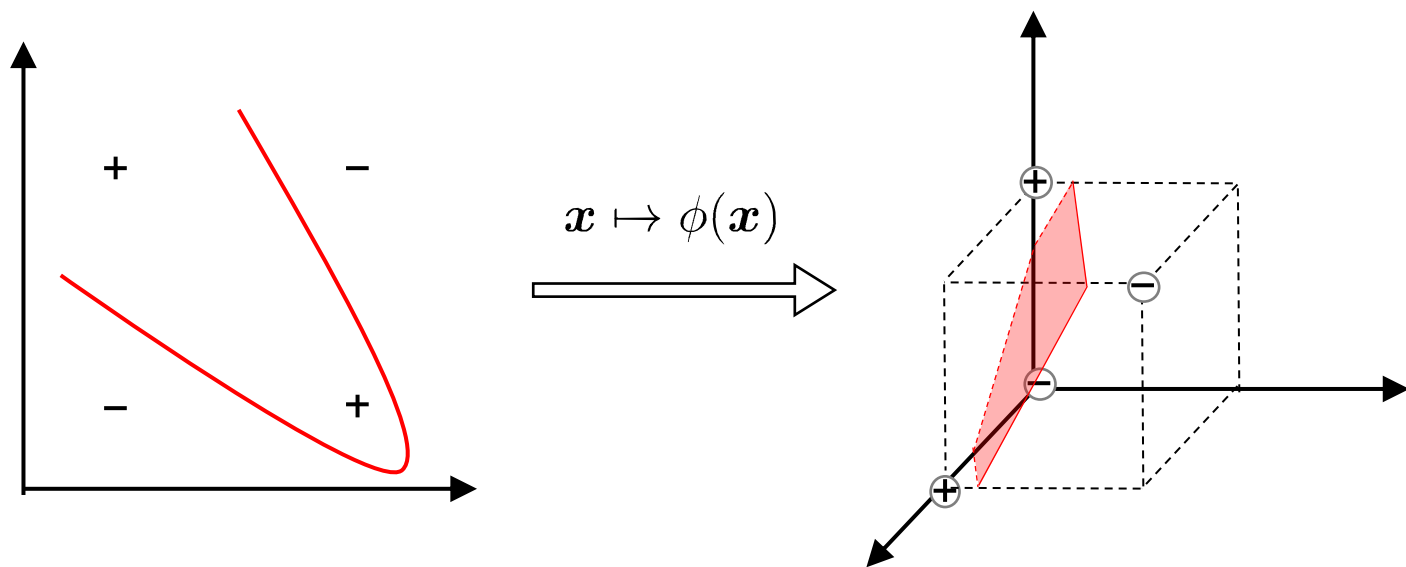
$$\mathbf{x}=(x_1,x_2)$$



线性不可分

□ 非线性映射

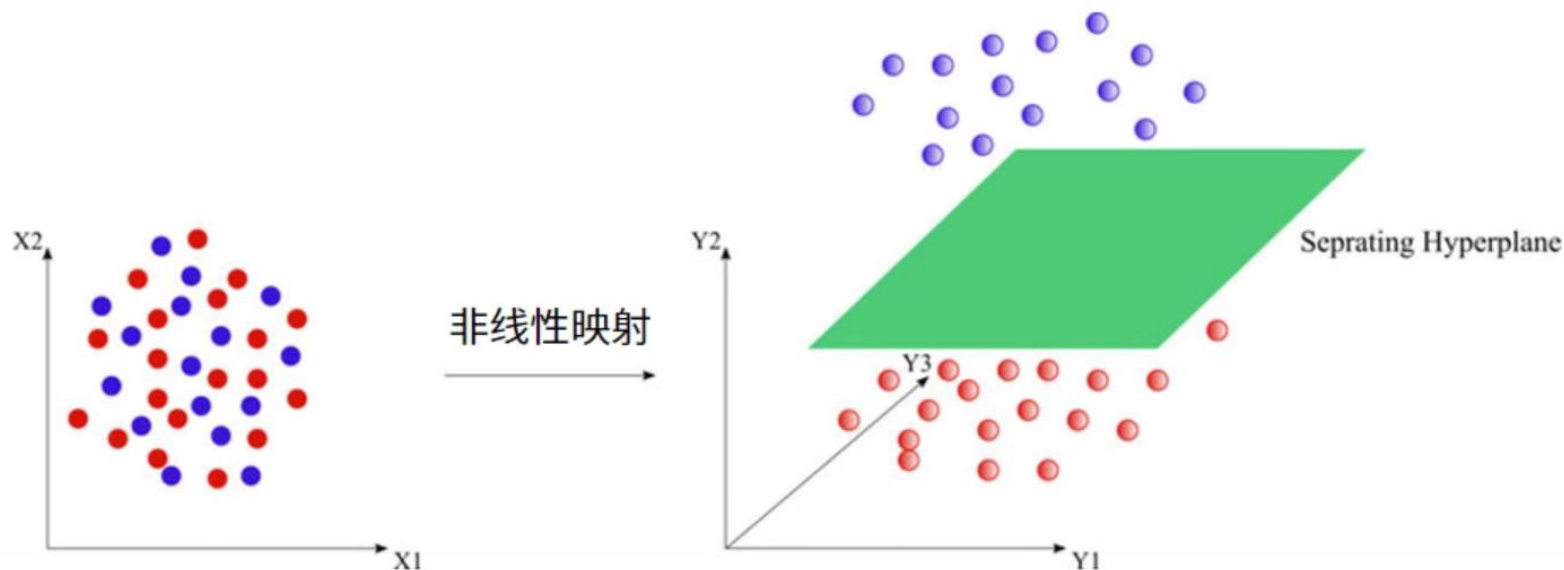
- 将样本从原始空间映射到一个更高维的特征空间，使得样本在高维空间内线性可分



线性不可分

□ 非线性映射

- 将样本从原始空间映射到一个更高维的特征空间，使得样本在高维空间内线性可分



核支持向量机

□ 设样本 $\mathbf{x}$ 映射后的向量为 $\phi(\mathbf{x})$, 划分超平面为 $f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b$.

原始问题

$$\begin{aligned} \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$

对偶问题

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j) - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

只以内积的形式出现

预测

$$f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b = \sum_{i=1}^m \alpha_i y_i \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}) + b$$

核函数

- 基本想法：避免直接将数据样本从低维投射到高维，再在高维空间线性划分，而是直接定义高维空间内积

$$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$$

- 直接计算高维函数的内积 $\phi(\mathbf{x}_i)^\top \phi(\mathbf{x}_j)$ 比较复杂
- 计算核函数 $K(\mathbf{x}_i, \mathbf{x}_j)$ 会相对比较容易

- 对偶问题：

$$\begin{aligned} \max_{\alpha} \quad & \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) \\ \text{s. t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \alpha_i \geq 0, i = 1, 2, \dots, m \end{aligned}$$

核函数

- 已知映射 $\phi(\cdot)$ 可得核函数 $K(\cdot, \cdot)$, 若映射 $\phi(\cdot)$ 未知, 什么函数可做核函数?
- **Mercer**定理(充分非必要): 只要一个对称函数所对应的核矩阵 $\mathbf{K}$ 半正定, 则它就能作为核函数来使用

$$\text{核矩阵 } \mathbf{K} = \begin{bmatrix} K(\mathbf{x}_1, \mathbf{x}_1), \dots, K(\mathbf{x}_1, \mathbf{x}_j), \dots, K(\mathbf{x}_1, \mathbf{x}_m) \\ \vdots \\ K(\mathbf{x}_i, \mathbf{x}_1), \dots, K(\mathbf{x}_i, \mathbf{x}_j), \dots, K(\mathbf{x}_i, \mathbf{x}_m) \\ \vdots \\ K(\mathbf{x}_m, \mathbf{x}_1), \dots, K(\mathbf{x}_m, \mathbf{x}_j), \dots, K(\mathbf{x}_m, \mathbf{x}_m) \end{bmatrix}$$

- 常用核函数:

名称	表达式	参数
线性核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$	
多项式核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j)^d$	$d \geq 1$ 为多项式的次数
高斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\delta^2}\right)$	$\delta > 0$ 为高斯核的带宽(width)
拉普拉斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ }{\delta}\right)$	$\delta > 0$
Sigmoid核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta \mathbf{x}_i^\top \mathbf{x}_j + \theta)$	$\tanh$ 为双曲正切函数, $\beta > 0, \theta < 0$

核函数的选择

□ 核函数选择

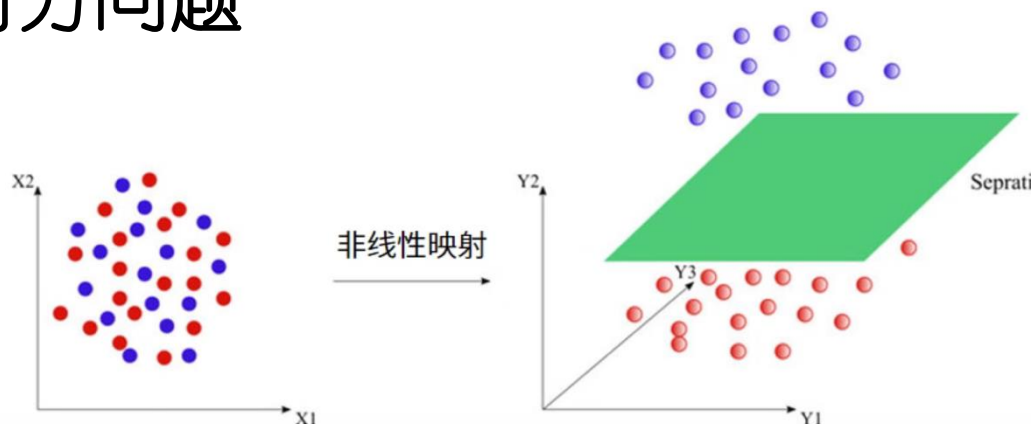
核函数选择成为支持向量机的最大变数，若核函数选择不合适，意味着将样本映射到不合适的特征空间中，很可能会导致模型性能不佳。

□ 核函数选择基本经验

- 文本数据：线性核
- 情况未知：高斯核

从“换空间”到“核技巧”——创新的本质

□ 线性不可分问题



□ 核方法的核心问题

- 显示映射: $x \rightarrow \phi(x)$
- 核函数: $K(x, y) \rightarrow \phi(x) * \phi(y)$

为什么不直接在高维空间计算？核技巧节省了什么？？

从“换空间”到“核技巧”——创新的本质

显示映射： $x \rightarrow \phi(x)$  核技巧： $K(x, y) \rightarrow \phi(x) * \phi(y)$



思维跃迁：

- 从“直接求解” → “变换问题空间”
- 从“增加复杂度” → “重构计算方式”



思考与领悟：

- ★ 创新源于对问题本质的深入理解
- ★ 从“换空间”到“核技巧”的思维跃迁
- ★ 真正的创新是方法论突破，而非简单叠加

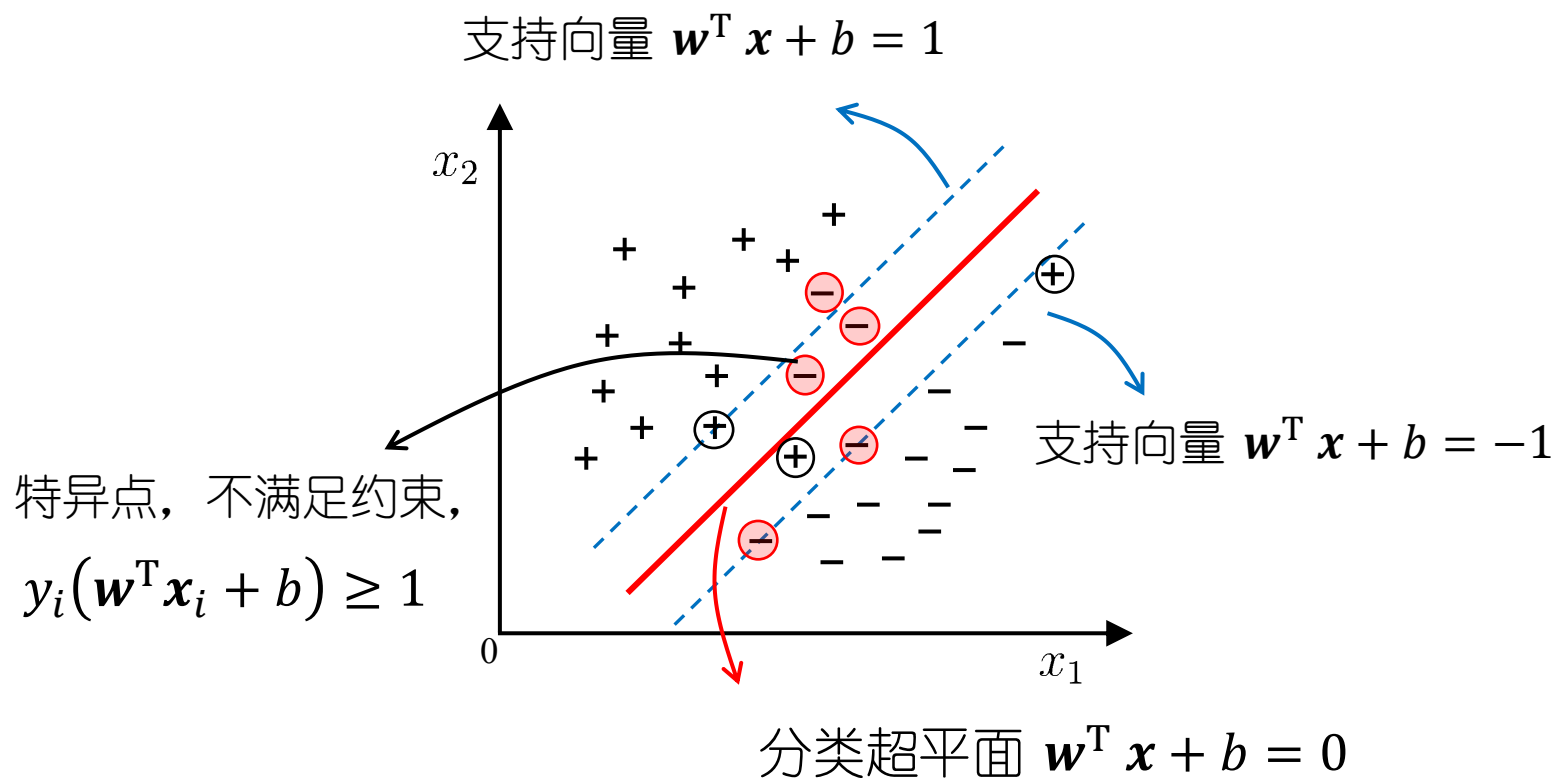
大纲

- 支持向量机基本型
- 对偶问题
- SMO算法（选讲）
- 核函数
- 软间隔与正则化
- 支持向量回归

软间隔

□ 面临的问题

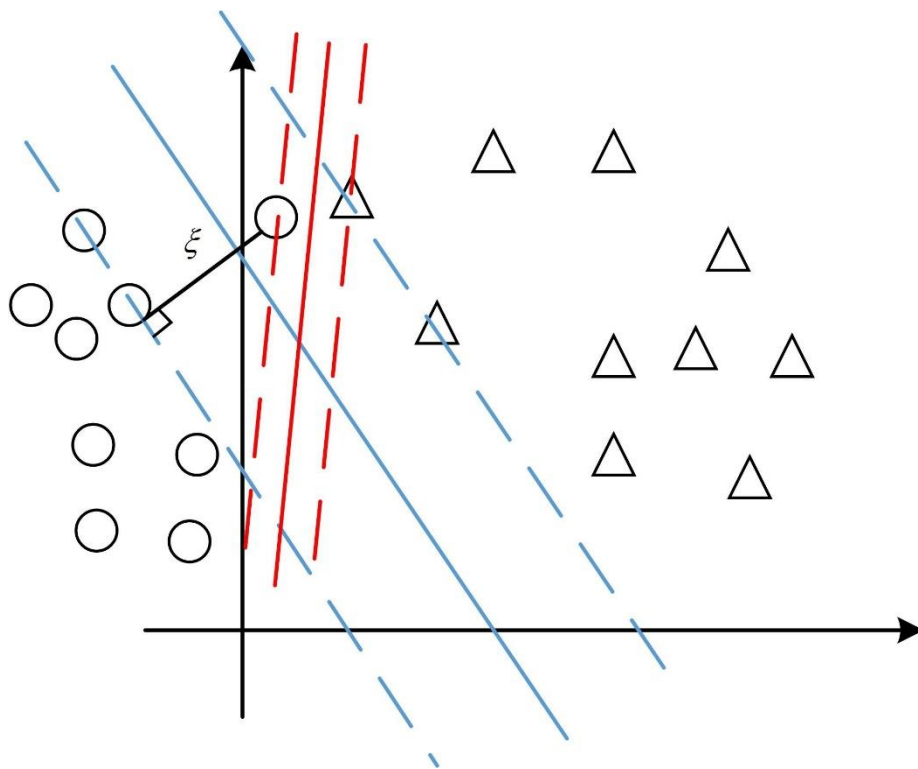
- 训练数据中存在一些特异点，不能满足函数间隔大于等于1的约束



软间隔

□ 面临的问题

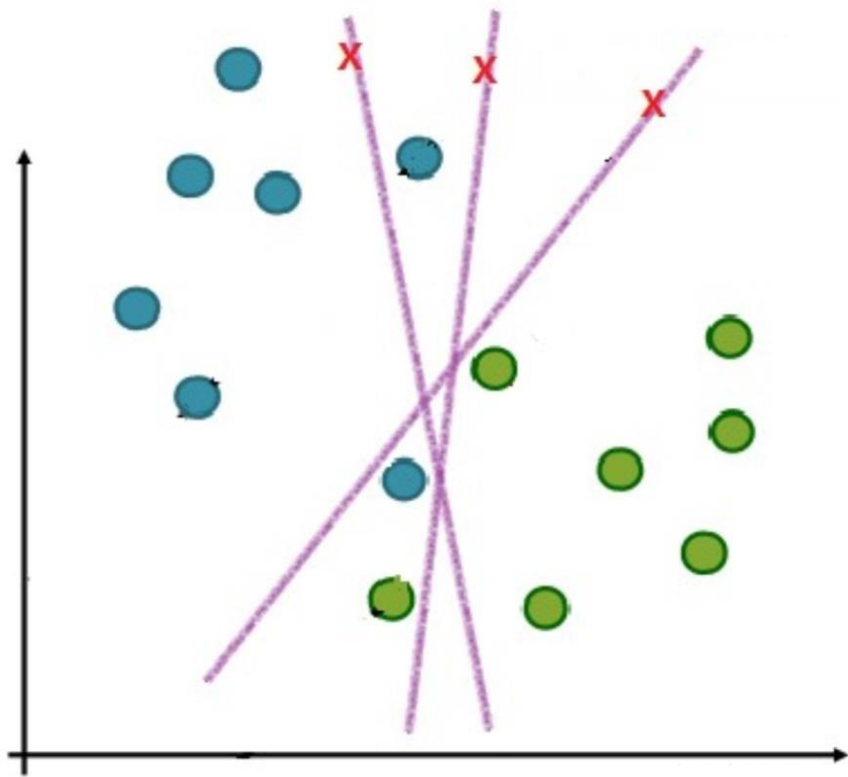
- 训练数据中存在一些特异点，不能满足函数间隔大于等于1的约束



软间隔

□ 面临的问题

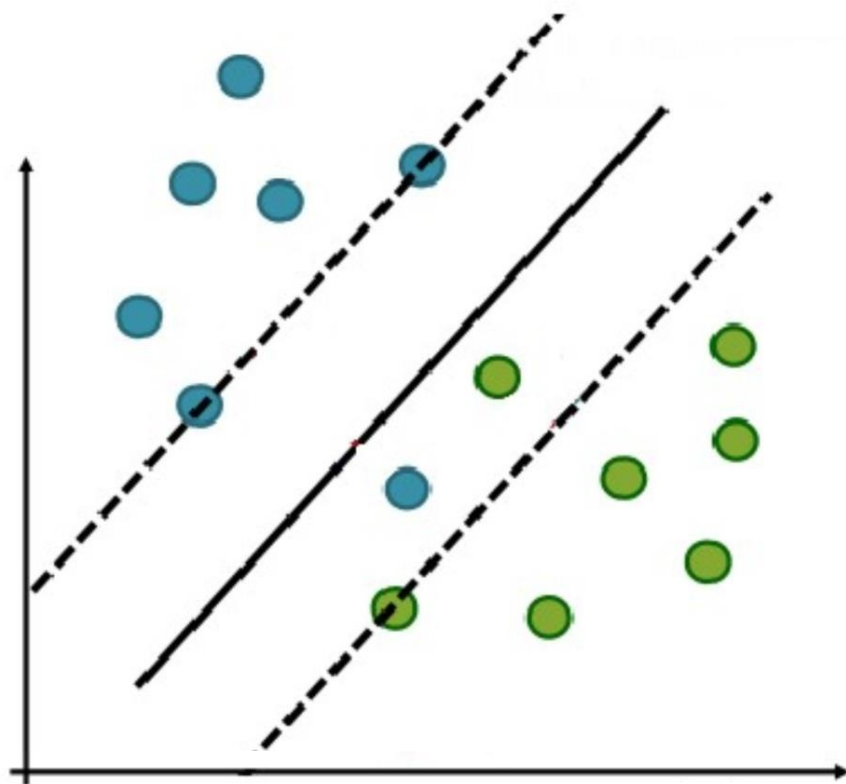
- 训练数据中存在一些特异点，不能满足超平面划分1的约束



软间隔

- 解决方案

允许部分样本点不满足约束条件： $1 - y_i(w^T x_i + b) \leq 0$



软间隔

□ 面临的问题

- 训练数据中存在一些特异点，不能满足函数间隔大于等于1的约束，即，

特异点满足： $1 - y_i(w^T x_i + b) \leq 0$

● 解决方案

- 引入“软间隔”，允许部分样本不满足约束，但在最大化间隔的同时，不满足约束的样本应该尽可能小。

- 优化目标

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m l_{0/1}(y_i(w^\top \phi(x_i) + b) - 1)$$

- 其中 $C > 0$ 是调和两个目标的惩罚参数， $l_{0/1}$ 是0/1损失函数

$$l_{0/1} = \begin{cases} 1 & z < 0 \\ 0 & otherwise \end{cases}$$

软间隔

- 特性

- 只有当约束不满足的情况下，损失函数 $l_{0/1} = 1$ ，最小化目标函数意味着希望不满足约束的样本越少越好；
- 惩罚参数C越大，意味着对误分类样本惩罚越大，越要求避免误分类样本。

- 存在问题

损失函数 $l_{0/1}$ 非凸、不连续，优化目标不易直接求解，需要利用数学性质较好的连续函数进行替代

替代损失

常用的三种损失函数为：

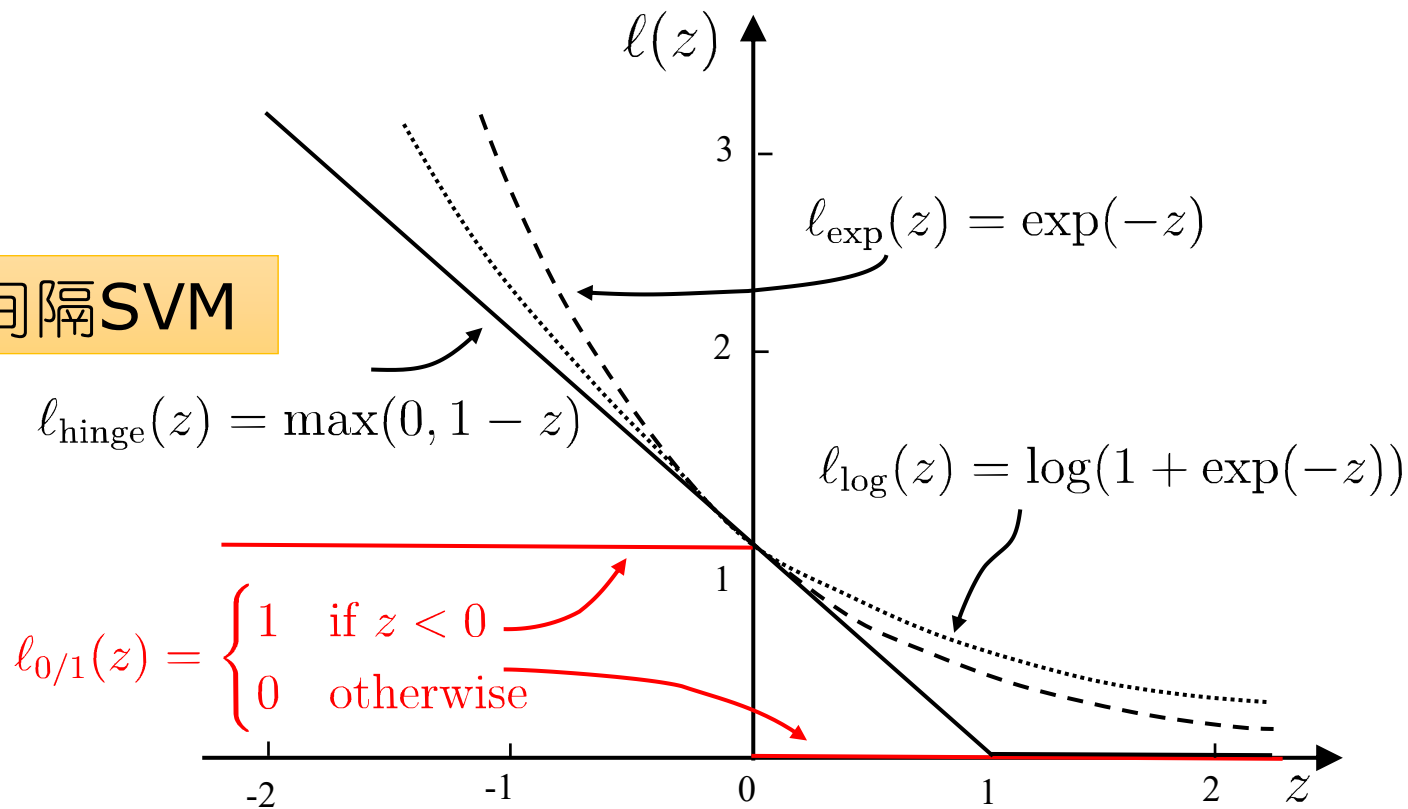
hinge 损失: $\ell_{\text{hinge}}(z) = \max(0, 1 - z)$

指数损失(exponential loss): $\ell_{\text{exp}}(z) = \exp(-z)$

对率损失(logistic loss): $\ell_{\text{log}}(z) = \log(1 + \exp(-z))$

替代损失

软间隔SVM



替代损失函数数学性质较好，一般是**0/1**损失函数的上界

软间隔支持向量机

- 利用hinge函数对原始问题进行转换

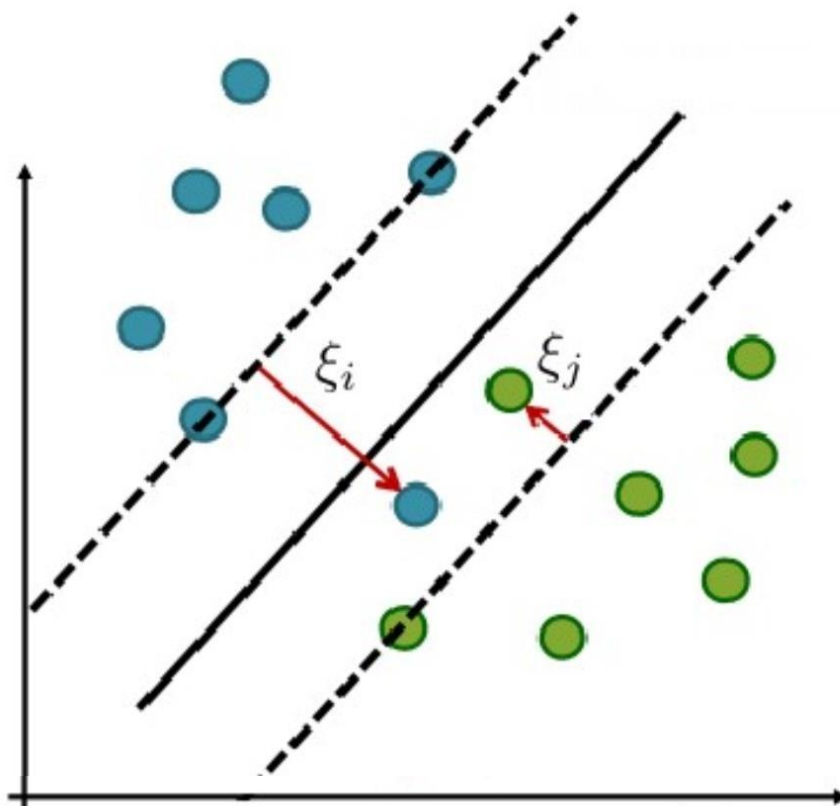
$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m l_{0/1} (y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) - 1)$$



$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \max(0, 1 - y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b))$$

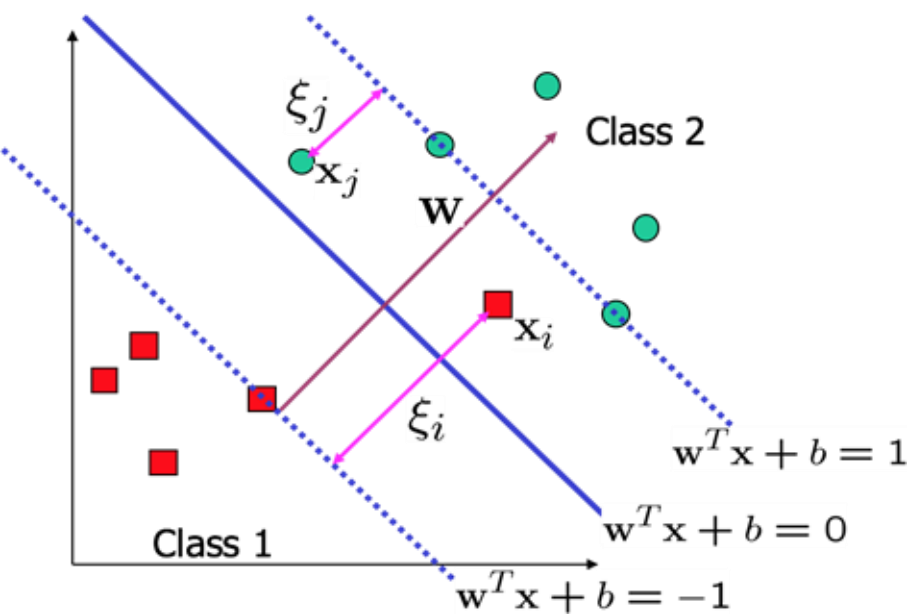
软间隔支持向量机

为了度量这个间隔软到何种程度，我们为每个样本引入一个松弛变量 ξ_i ，令 $\xi_i \geq 0$ ，且 $1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) - \xi_i \leq 0$ 。对应如下图所示：



软间隔支持向量机

为了度量这个间隔软到何种程度，我们为每个样本引入一个松弛变量 ξ_i ，令 $\xi_i \geq 0$ ，且 $1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) - \xi_i \leq 0$ 。对应如下图所示：



新的目标函数：

$$\frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i$$

C ：选择的误差和最大间隔之间的权衡参数；

C 越大，表示对错误的惩罚越高。

软间隔支持向量机

- 引入松弛变量 $\xi_i \geq 0$ ，优化目标转变为

$$\min_{w, b, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

$$s. t. \quad g(w, b) = 1 - y_i(\mathbf{w}^T \mathbf{x}_i + b) - \xi_i \leq 0,$$

$$\xi_i \geq 0, i = 1, 2, \dots, m$$

软间隔支持向量机

- 针对不等式约束分别引入拉格朗日乘子 $\alpha_i \geq 0, \mu_i \geq 0$
- 优化目标拉格朗日函数

$$L(\mathbf{w}, b, \xi, \alpha, \mu) = \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i + \sum_{i=1}^m \alpha_i (1 - \xi_i - y_i(\mathbf{w}^T \mathbf{x}_i + b)) - \sum_{i=1}^m \mu_i \xi_i$$

- 分别对变量 $\mathbf{w}, b, \xi_i$ 偏导置零

$$\mathbf{w} = \sum_{i=1}^m \alpha_i y_i \mathbf{x}_i \quad 0 = \sum_{i=1}^m \alpha_i y_i \quad C = \alpha_i + \mu_i$$

软间隔支持向量机

➤ 对偶问题

$$\max_{\alpha} \quad \sum_{i=1}^m \alpha_i - \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$$

$$s. t. \quad \sum_{i=1}^m \alpha_i y_i = 0,$$

$$0 \leq \alpha_i \leq C, i = 1, 2, \dots, m$$

软间隔支持向量机

➤ KKT条件（软间隔）

$$\begin{cases} \alpha_i \geq 0, & \mu_i \geq 0 \\ y_i f(\mathbf{x}_i) - 1 + \xi_i \geq 0 \\ \alpha_i (y_i f(\mathbf{x}_i) - 1 + \xi_i) = 0 \\ \xi_i \geq 0, & \mu_i \xi_i = 0 \end{cases}$$

➤ KKT条件（硬间隔）

$$\begin{cases} \alpha_i \geq 0 \\ f(\mathbf{x}) y_i - 1 \geq 0 \\ \alpha_i (f(\mathbf{x}) y_i - 1) = 0 \end{cases}$$

软间隔支持向量机

➤ 稀疏性

- 对于任意样本 $(\mathbf{x}_i, y_i)$ ，总 $\alpha_i = 0$ 有或 $y_i f(\mathbf{x}_i) = 1 - \xi_i$
- 若 $\alpha_i = 0$ ，则样本不对模型 $f(\mathbf{x})$ 产生任何影响
- 若 $\alpha_i = 0$ ，则 $y_i f(\mathbf{x}_i) = 1 - \xi_i$ ，则该样本是支持向量
- 若 $\alpha_i < C$ ，则 $\mu_i > 0$ ，则 $\xi_i = 0$ ，样本落在最大间隔上
- 若 $\alpha_i = C$ ，则 $\mu_i = 0$ ，此时若 $\xi_i \leq 1$ ，样本落在最大间隔内部，
若 $\xi_i > 1$ ，样本被错误分类。

正则化

- 支持向量机学习模型的更一般形式

$$\min_f \Omega(f) + C \sum_{i=1}^m l(f(\mathbf{x}_i), y_i)$$



结构风险, 描述模型的某些性质



经验风险, 描述模型与训练数据的契合程度

- 通过替换上面两个部分, 可以得到其他学习模型

- 对数几率回归(Logistic Regression)

$$\max \sum_{i=1}^m y_i (\mathbf{w}^T \mathbf{x}_i + b) - \ln(1 + \exp(\mathbf{w}^T \mathbf{x}_i + b))$$

$$\min \sum_{i=1}^m -y_i (\mathbf{w}^T \mathbf{x}_i + b) + \ln(1 + \exp(\mathbf{w}^T \mathbf{x}_i + b))$$

- 最小绝对收缩选择算子(LASSO)

第二章 欠拟合和过拟合

□过拟合 (overfitting) :

- 模型过度拟合了训练数据，表现为模型在训练数据上表现很好，但在测试数据上表现很差。
- 学习器把训练样本学习的“太好”，将训练样本本身的特性当做所有样本的共性，导致泛化性能下降。
- 过拟合的主要原因有：
 1. 参数太多，模型复杂度过高；
 2. 建模样本选取有误，导致选取的样本数据不足以代表预定的分类规则；
 3. 样本噪音干扰过大，使得机器将部分噪音认为是特征从而扰乱了预设的分类规则；
 4. 假设的模型无法合理存在，或者说是假设成立的条件实际并不成立。

过拟合

□ 过拟合(overfitting):

为了避免欠拟合和过拟合，可以采取以下方法：

- 减少模型的复杂度，例如减小神经网络的层数或节点数，
- 使用正则化方法（如L1和L2正则化），
- 增加训练数据量，
- 使用数据增强等技术，
- 采用early stop策略（早停策略）。

正则化

- 正则化通过在目标函数中加入一个“惩罚项”来控制模型复杂度，使得模型不至于拟合得太过极端。

目标函数一般变成如下形式：

$$\text{Loss} = \text{原始损失} + \lambda \cdot \text{正则化项}$$

例如：

$$\min_f \Omega(f) + C \sum_{i=1}^m l(f(\mathbf{x}_i), y_i)$$



结构风险，描述模型的某些性质



经验风险，描述模型与训练数据的契合程度

第二章 过拟合

□正则化 (overfitting) :

➤ 正则化：在目标损失函数中增加用来描述网络复杂程度的正则化项，例如连接权值与阈值的平方和，则：

则误差目标函数可改变为：

$$E = \lambda \underbrace{\frac{1}{m} \sum_{k=1}^m E_k}_{\text{平均累计误差}} + (1 - \lambda) \underbrace{\sum_i w_i^2}_{\text{权值和阈值的“复杂程度”}}$$

其中 $\lambda \in (0, 1)$ ，用于对“经验误差”和“网络复杂度”这两项进行折中；常通过交叉验证法进行估计。

第二章 过拟合

□正则化 (overfitting) :

➤ 什么参数越小代表模型越简单？

因为参数的稀疏，在一定程度上实现了特征的选择。

越复杂的模型，越是会尝试对所有的样本进行拟合，甚至包括一些异常样本点，这就容易造成在较小的区间里预测值产生较大的波动，这种较大的波动也反映了在这个区间里的导数很大，而只有较大的参数值才能产生较大的导数。因此复杂的模型，其参数值会比较大。

因此参数越少代表模型越简单。

测试题1

3. 以自己的理解，阐述什么是过拟合和欠拟合？

□ 欠拟合(underfitting)：

模型无法拟合训练数据，表现为模型的预测能力较差。

这通常发生在模型过于简单或者数据特征复杂时。如：线性模型拟合非线性数据。

为了避免欠拟合和过拟合，可以采取以下方法：

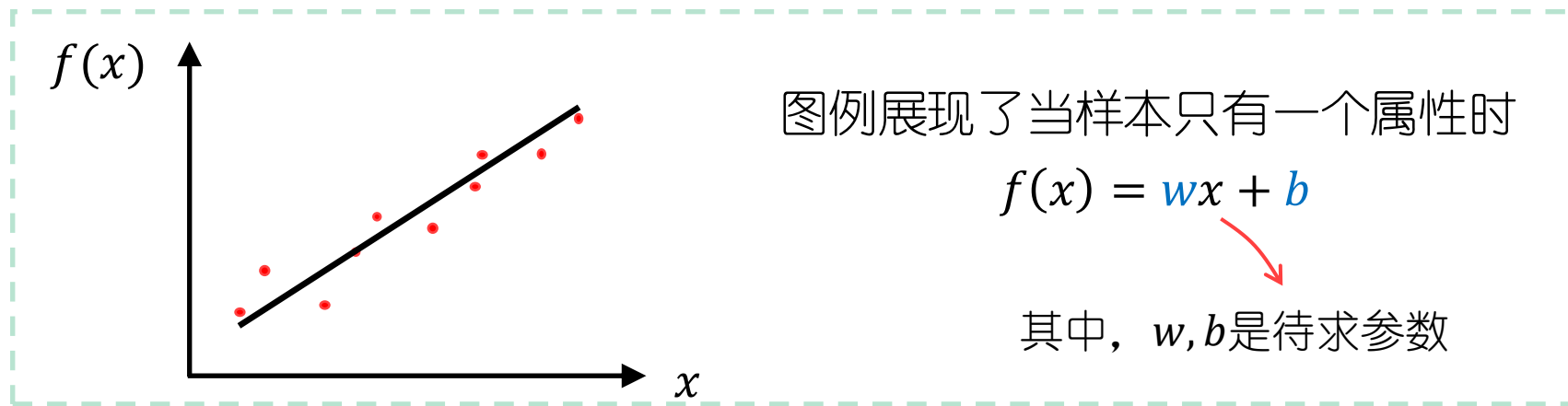
- 使用更复杂的模型，如决策树或随机森林。而针对决策树：拓展分支
- 增加模型的复杂度：增加神经网络的层数或节点数

大纲

- 支持向量机基本型
- 对偶问题
- SMO算法（选讲）
- 核函数
- 软间隔与正则化
- 支持向量回归

线性模型

通过对数据样本属性的线性组合来进行预测的函数

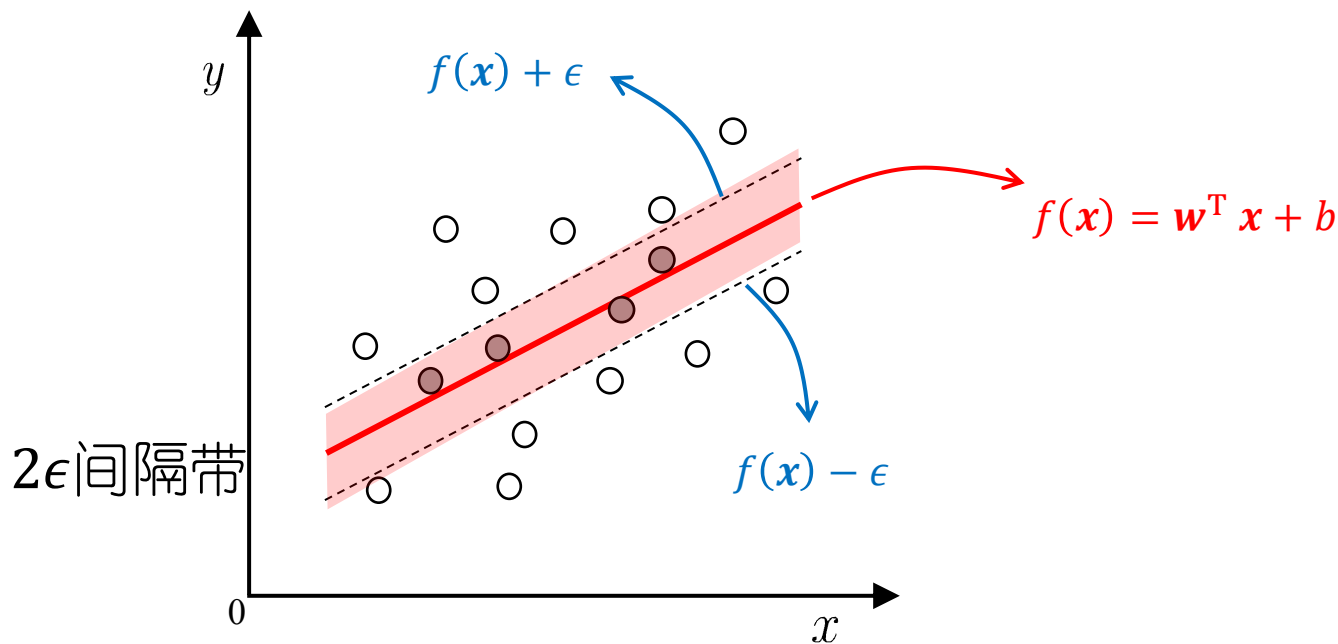


	区别	损失函数
线性回归	应用：回归问题 目的：找出最优拟合 输出是一个连续值	最小化平方误差： $L = \frac{1}{2m} \sum_{i=1}^m (f(x_i) - y_i)^2$

支持向量回归SVR

- 回归问题

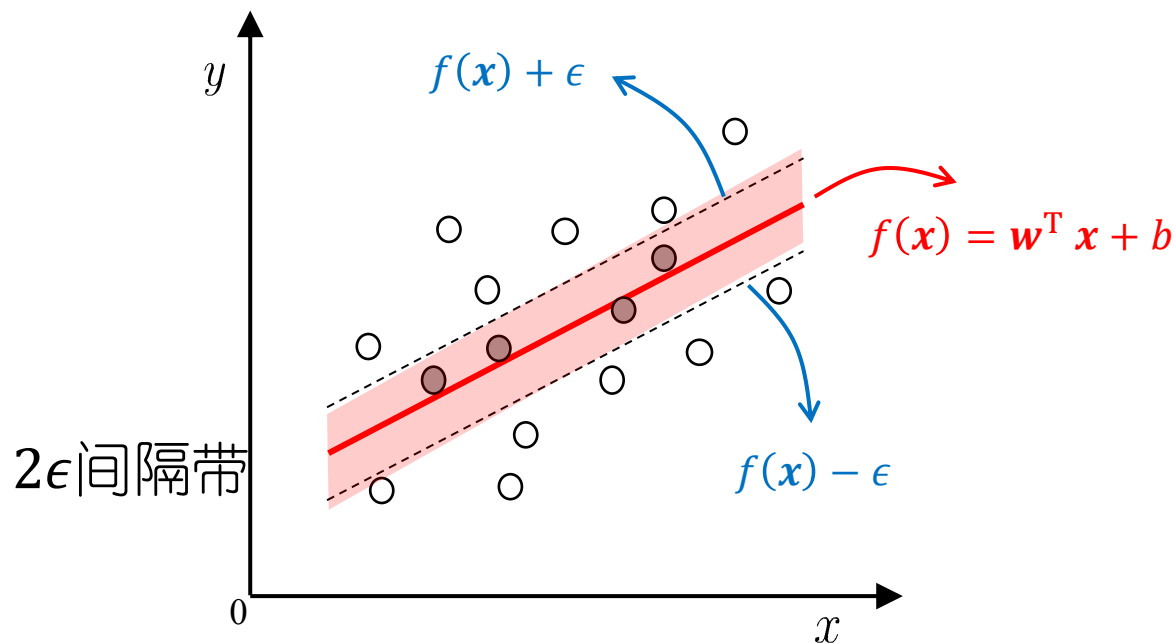
不同于传统回归模型基于输出 $f(x)$ 与真实输出 y 之间的差别来计算损失，SVR允许模型输出和实际输出间存在 2ϵ 的偏差



SVR问题形式
$$\min_{w, b, \xi_i} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^m \ell_{\epsilon}(f(x_i) - y_i)$$

支持向量回归SVR

- 优化目标



优化目标

$$\min_{w, b, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2$$

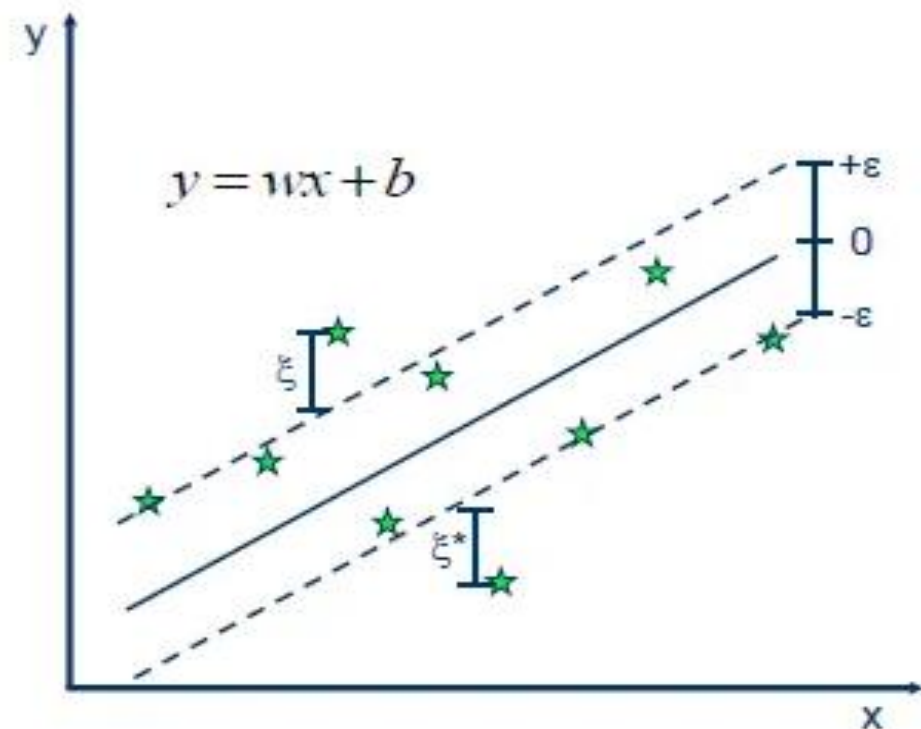
限制条件:

$$y_i - \mathbf{w}^\top \mathbf{x}_i - b \leq \varepsilon$$

$$\mathbf{w}^\top \mathbf{x}_i + b - y_i \leq \varepsilon$$

支持向量回归SVR

- 优化目标



优化目标

$$\min_{w, b, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \ell_{\epsilon}(f(\mathbf{x}_i) - y_i)$$

限制条件:

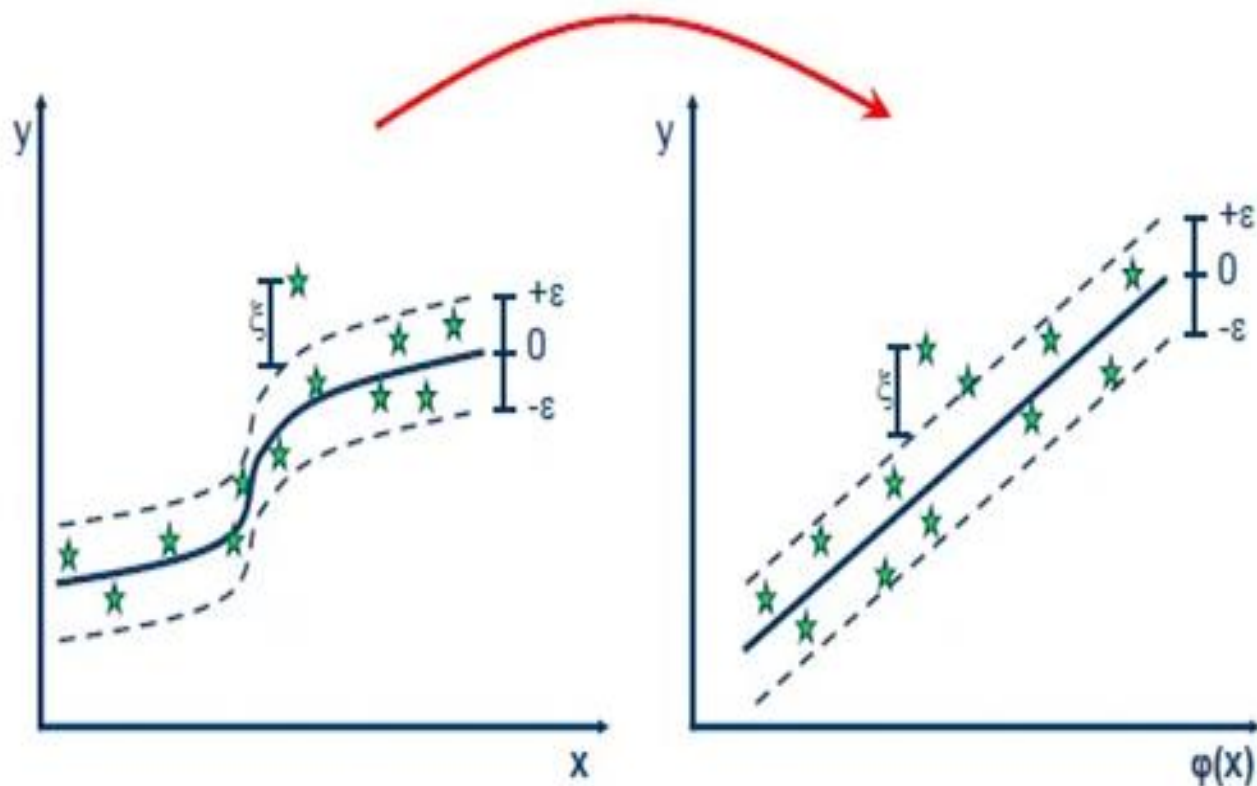
$$y_i - \mathbf{w}^{\top} \mathbf{x}_i - b \leq \epsilon + \xi_i$$

$$\mathbf{w}^{\top} \mathbf{x}_i + b - y_i \leq \epsilon + \xi_i^*$$

$$\xi_i, \xi_i^* \geq 0$$

支持向量回归SVR

- 优化目标

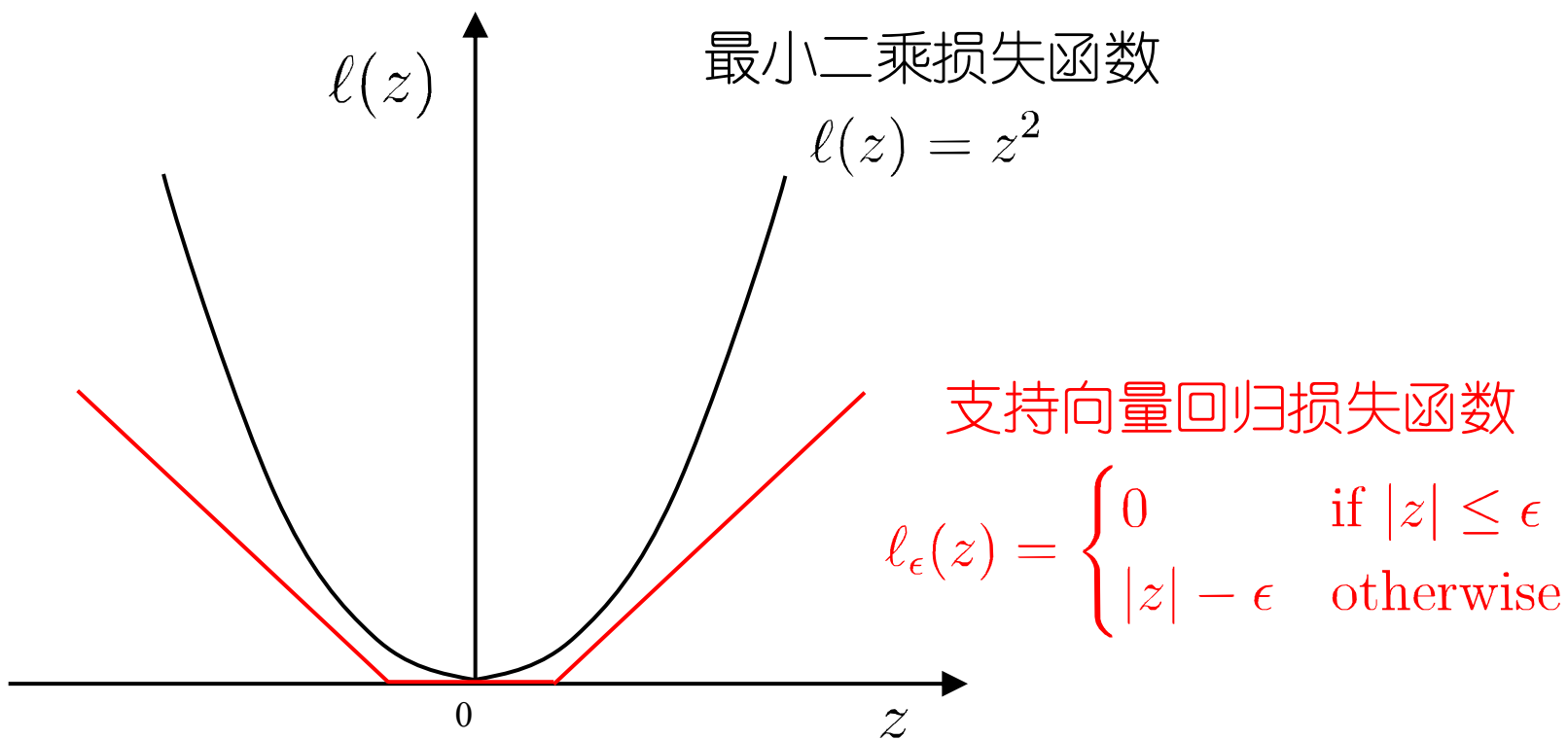


$$y = \sum_{i=1}^N (\alpha_i - \alpha_i^*) \cdot \langle \varphi(x_i), \varphi(x) \rangle + b$$

$$y = \sum_{i=1}^N (\alpha_i - \alpha_i^*) \cdot K(x_i, x) + b$$

损失函数

- **SVR特性**：落入中间 2ϵ 间隔带的样本不计算损失，从而使得模型获得稀疏性



形式化

- 引入松弛变量，原始问题转变为

$$\begin{aligned} \min_{\mathbf{w}, b, \xi_i, \hat{\xi}_i} \quad & \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m (\xi_i + \hat{\xi}_i) \\ \text{s.t.} \quad & y_i - \mathbf{w}^\top \phi(\mathbf{x}_i) - b \leq \epsilon + \xi_i, \\ & y_i - \mathbf{w}^\top \phi(\mathbf{x}_i) - b \geq -\epsilon - \hat{\xi}_i, \\ & \xi_i \geq 0, \hat{\xi}_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

- 对偶问题

$$\begin{aligned} \min_{\alpha, \hat{\alpha}} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m (\alpha_i - \hat{\alpha}_i)(\alpha_j - \hat{\alpha}_j) \kappa(\mathbf{x}_i, \mathbf{x}_j) + \sum_{i=1}^m (\alpha_i \epsilon - y_i) + \hat{\alpha}_i (\epsilon + y_i) \\ \text{s.t.} \quad & \sum_{i=1}^m (\alpha_i - \hat{\alpha}_i) = 0, \\ & 0 \leq \alpha_i \leq C, \quad 0 \leq \hat{\alpha}_i \leq C. \end{aligned}$$

- SVR模型

$$f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}) + b = \sum_{i=1}^m (\hat{\alpha}_i - \alpha_i) y_i \kappa(\mathbf{x}_i, \mathbf{x}) + b$$

成熟的SVM软件包

- LIBSVM

<http://www.csie.ntu.edu.tw/~cjlin/libsvm/>

- LIBLINEAR

<http://www.csie.ntu.edu.tw/~cjlin/liblinear/>

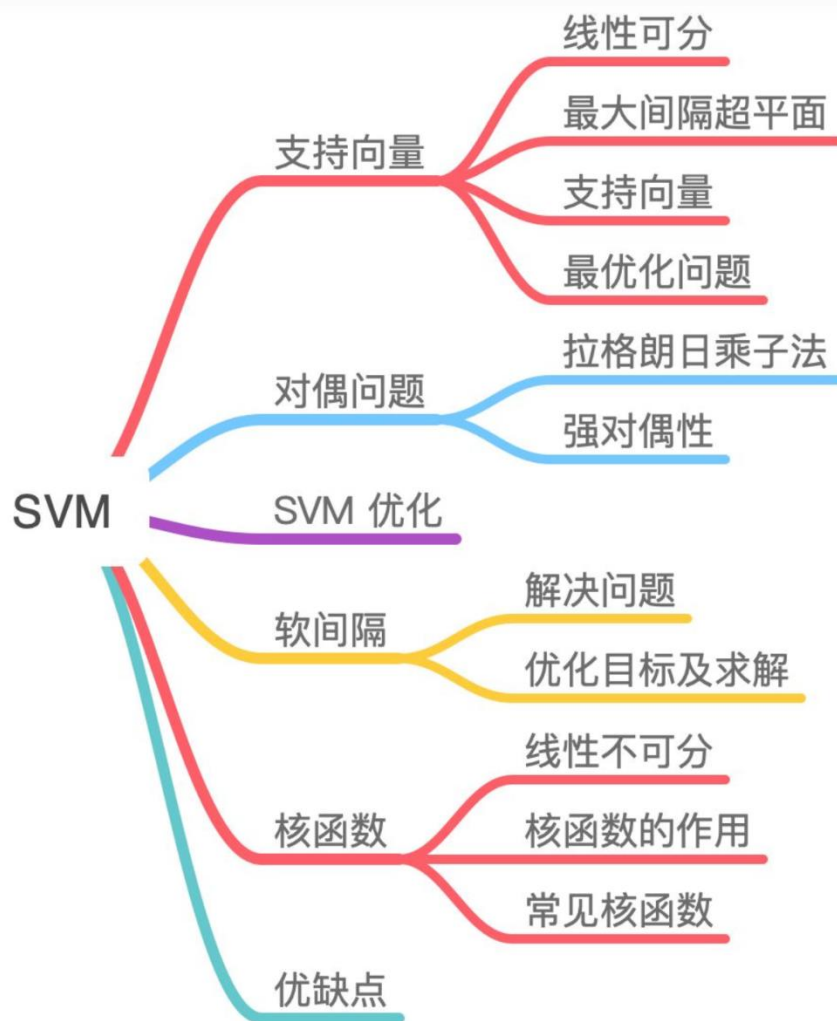
- SVM^{light}、SVM^{perf}、SVM^{struct}

http://svmlight.joachims.org/svm_struct.html

- Pegasos

<http://www.cs.huji.ac.il/~shais/code/index.html>

总结



总结：SVM算法推导

□ 构造并求解约束最优化问题

目标：找到最大间隔划分超平面 $\mathbf{w}^\top \mathbf{x} + b = 0$

最优化目标

$$\begin{aligned} \arg \min_{\mathbf{w}, b} \quad & \frac{1}{2} \|\mathbf{w}\|^2 \\ \text{s.t.} \quad & y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, 2, \dots, m. \end{aligned}$$



对偶问题

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \quad \alpha_i \geq 0, \quad i = 1, 2, \dots, m. \end{aligned}$$

总结：SVM算法推导

□ 求得最优解 α_i^* ：（SMO算法求解）

□ 计算平面的权重和偏移项：

$$\mathbf{w}^* = \sum_{i=1}^m \alpha_i^* y_i \mathbf{x}_i \qquad b = \frac{1}{|S|} \sum_{s \in S} (y_s - \mathbf{w}^* \mathbf{x}_s)$$

□ 求得划分超平面：

$$\mathbf{w}^* \mathbf{x} + b = 0$$

□ 分类决策函数：

$$f(\mathbf{x}) = \text{sign}(\mathbf{w}^* \mathbf{x} + b)$$

总结：软间隔

最优化目标

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m l_{0/1} (y_i(\mathbf{w}^\top \phi(\mathbf{x}_i) + b) - 1)$$



引入松弛变量 $\xi_i \geq 0$

$$\min_{\mathbf{w}, b, \xi_i} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i$$

$$s. t. \quad g(\mathbf{w}, b) = 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b) - \xi_i \leq 0,$$



对偶问题

$$\min_{\alpha} \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j \mathbf{x}_i^\top \mathbf{x}_j - \sum_{i=1}^m \alpha_i$$

$$s. t. \sum_{i=1}^m \alpha_i y_i = 0, \quad 0 \leq \alpha_i \leq C, \quad i = 1, 2, \dots, m$$

总结：核函数

- 将样本 x 映射到高维空间 $\phi(x)$ ，使得样本在高维空间内线性可分，划分超平面为 $f(x) = w^T \phi(x) + b$.
 - 核函数 $\kappa(\mathbf{x}_i, \mathbf{x}_j)$ ：可以在低维空间进行高纬度映射之后的计算
- 最优化目标的对偶问题：

$$\begin{aligned} \min_{\alpha} \quad & \frac{1}{2} \sum_{i=1}^m \sum_{j=1}^m \alpha_i \alpha_j y_i y_j K(\mathbf{x}_i, \mathbf{x}_j) - \sum_{i=1}^m \alpha_i \\ \text{s.t.} \quad & \sum_{i=1}^m \alpha_i y_i = 0, \alpha_i \geq 0, i = 1, 2, \dots, m \end{aligned}$$

总结：核函数

□ 求得最优解 α_i^* ：（SMO算法求解）

□ 计算平面的权重和偏移项：

$$\mathbf{w}^* = \sum_{i=1}^m \alpha_i^* y_i \phi(\mathbf{x}_i) \quad b = \frac{1}{|S|} \sum_{s \in S} (y_s - \mathbf{w}^* \mathbf{x}_s)$$

□ 求得划分超平面：

$$f(\mathbf{x}) = \mathbf{w}^* \boldsymbol{\phi}(\mathbf{x}) + b = 0$$

$$\sum_{i=1}^m \alpha_i^* y_i K(\mathbf{x}_i, \mathbf{x}_j) + b = 0$$

总结：核函数

□ 常用的核函数

名称	表达式	参数
线性核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^\top \mathbf{x}_j$	
多项式核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^\top \mathbf{x}_j)^d$	$d \geq 1$ 为多项式的次数
高斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ ^2}{2\delta^2}\right)$	$\delta > 0$ 为高斯核的带宽(width)
拉普拉斯核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\frac{\ \mathbf{x}_i - \mathbf{x}_j\ }{\delta}\right)$	$\delta > 0$
Sigmoid核	$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \tanh(\beta \mathbf{x}_i^\top \mathbf{x}_j + \theta)$	$\tanh$ 为双曲正切函数, $\beta > 0, \theta < 0$

□ 核函数选择基本经验

- 文本数据：线性核
- 情况未知：高斯核（需要调参数 σ ）

注： σ 控制高斯核函数的作用范围，其值越大，高斯核函数的局部影响范围就越大；

σ 不要选太小，否则在分类任务中容易过拟合。

总结：SVM模型分类

□ 由简至繁的SVM模型包括：

- 当训练样本线性可分时，通过硬间隔最大化，学习一个线性可分支持向量机；
- 当训练样本近似线性可分时，通过软间隔最大化，学习一个线性支持向量机；
- 当训练样本线性不可分时，通过核技巧和软间隔最大化，学习一个非线性支持向量机；

总结：SVM模型分类

□ 由简至繁的SVM模型包括：

- 当训练样本线性可分时，通过硬间隔最大化，学习一个线性可分支持向量机；
- 当训练样本近似线性可分时，通过软间隔最大化，学习一个线性支持向量机；
- 当训练样本线性不可分时，通过核技巧和软间隔最大化，学习一个非线性支持向量机；

优点

- 有严格的数学理论支持，可解释性强，不依靠统计方法，从而简化了通常的分类和回归问题；
- 能找出对任务至关重要的关键样本（即：支持向量）；
- 采用核技巧之后，可以处理非线性分类/回归任务；
- 最终决策函数只由少数的支持向量所确定，计算的复杂性取决于支持向量的数目，而不是样本空间的维数，这在某种意义上避免了“维数灾难”。

缺点

- ❑ 训练时间长。当采用 SMO 算法时，由于每次都需要挑选一对参数，因此时间复杂度为 $O(N^2)$ ，其中 N 为训练样本的数量；
- ❑ 当采用核技巧时，如果需要存储核矩阵，则空间复杂度为 $O(N^2)$ ；模型预测时，预测时间与支持向量的个数成正比。
- ❑ 当支持向量的数量较大时，预测计算复杂度较高。因此支持向量机目前只适合小批量样本的任务，无法适应百万甚至上亿样本的任务。

应用例子

□ 如何在Python中实现SVM ?

#导入库

```
from sklearn import svm
```

#假设您有用于训练数据集的X(特征数据)和Y(目标)，以及测试数据的x_test(特征数据)

#创建SVM分类对象

```
model = svm.svc(kernel='linear', c=1, gamma=1)
```

#与之相关的选项有很多，比如更改kernel值（内核）、gamma值和C值。使用训练集训练模型，并检查成绩

```
model.fit(X, y)
```

```
model.score(X, y)
```

#预测输出

```
predicted= model.predict(x_test)
```

应用例子

□ 如何调整SVM的参数？

- SVM可用的参数列表

```
sklearn.svm.SVC(C=1.0, kernel='rbf', degree=3, gamma=0.0, coef0=0.0,  
shrinking=True, probability=False, tol=0.001, cache_size=200,  
class_weight=None, verbose=False, max_iter=-1, random_state=None)
```

- 对模型性能影响较大的重要参数，如 “kernel”， “gamma” 和 “C”。

kernel: 内核选项有 “linear”， “rbf”， “poly” 等（“rbf” 为高斯核函数）；

其中 “rbf” 和 “poly” 对于找到非线性超平面是很有用的。

gamma: ‘rbf’， ‘poly’ 和 ‘sigmoid’ 的**内核系数**。其值越高，则会根据训练数据集进行精确拟合，也就是泛化误差从而导致过拟合问题。

C: 误差项的**惩罚参数**C，控制了平滑决策边界与正确分类训练点之间的权衡。

应用例子：使用linear的内核

示例：使用linear的内核

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm, datasets

#导入数据
iris = datasets.load_iris()
X = iris.data[:, :2] #我们可以只考虑前两个特征
#我们可以使用双数据集来避免丑陋的切片
y = iris.target

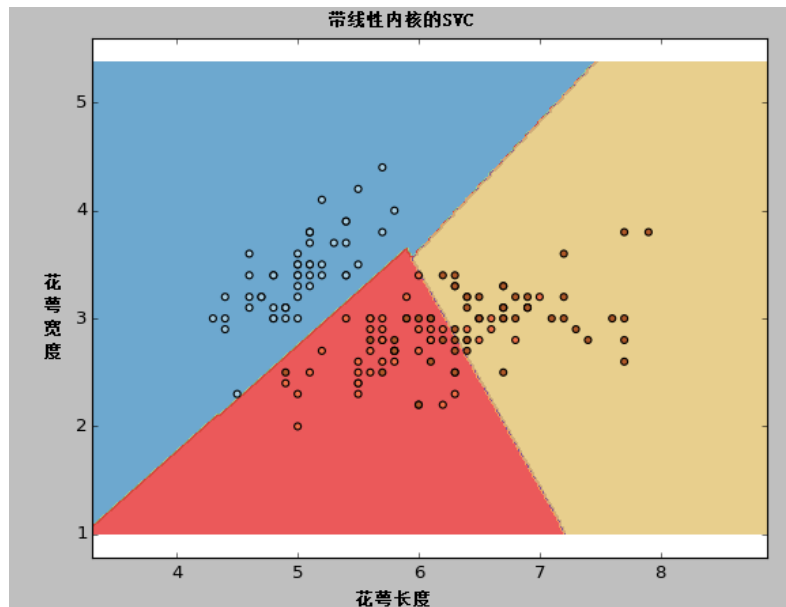
#我们创建了一个SVM实例并对数据进行拟合。不进行缩放
#是因为我们想要画出支持向量
C = 1.0 #SVM正则化参数
svc = svm.SVC(kernel='linear', C=1, gamma=0).fit(X, y)

#创建一个网格来进行可视化
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
h = (x_max / x_min)/100
xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
np.arange(y_min, y_max, h))
```

应用例子：使用**linear**的内核

```
plt.subplot(1, 1, 1)
Z = svc.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=plt.cm.Paired, alpha=0.8)

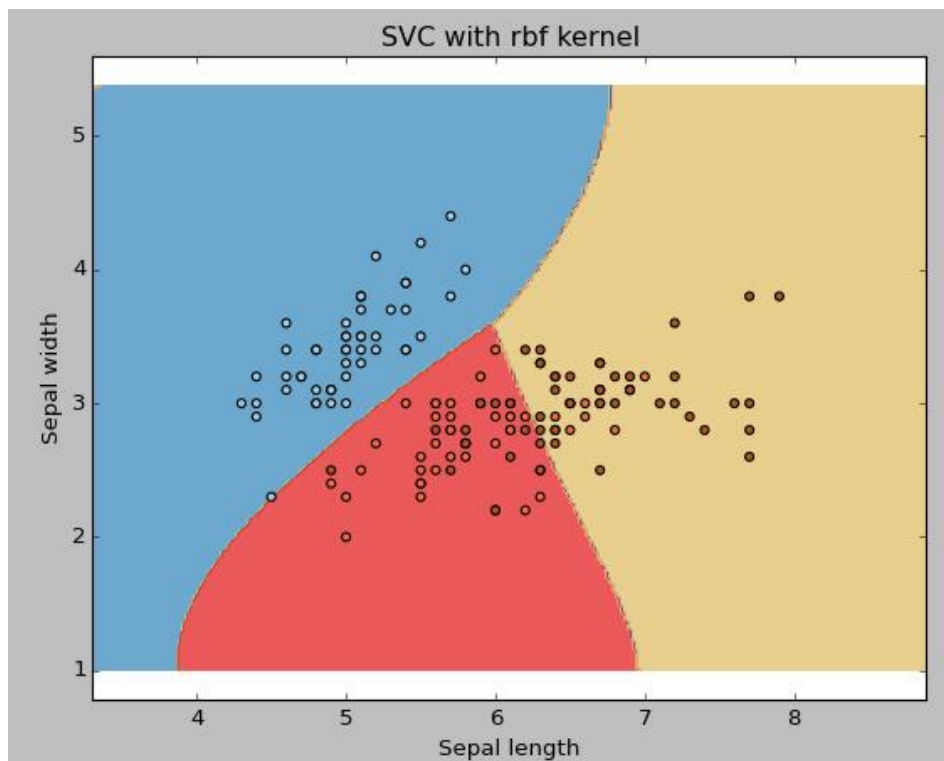
plt.scatter(X[:, 0], X[:, 1], c=y, cmap=plt.cm.Paired)
plt.xlabel('Sepal length')
plt.ylabel('Sepal width')
plt.xlim(xx.min(), xx.max())
plt.title('SVC with linear kernel')
plt.show()
```



应用例子： 使用RBF的内核

示例：使用RBF的内核 (RBF: Radial basis function, 径向基或高斯核函数)

将例子中 “`svc = svm.SVC(kernel= 'linear' , C=1,gamma=0).fit(X, y)`” 替换成
“`svc = svm.SVC (kernel = 'rbf' , C = 1, gamma = 0) .fit (X, y)`” ,
其他不变。



应用例子： 使用RBF的内核

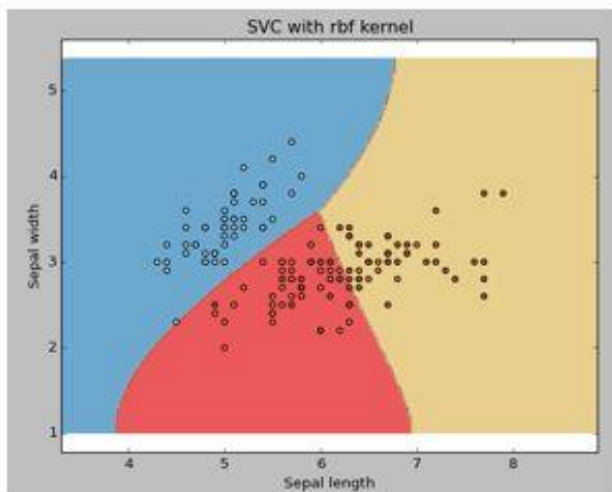
示例：使用不同的伽玛值，如0、10或100时，

```
svc = svm.SVC (kernel = 'rbf' , C = 1, gamma = 0) .fit (X, y)
```

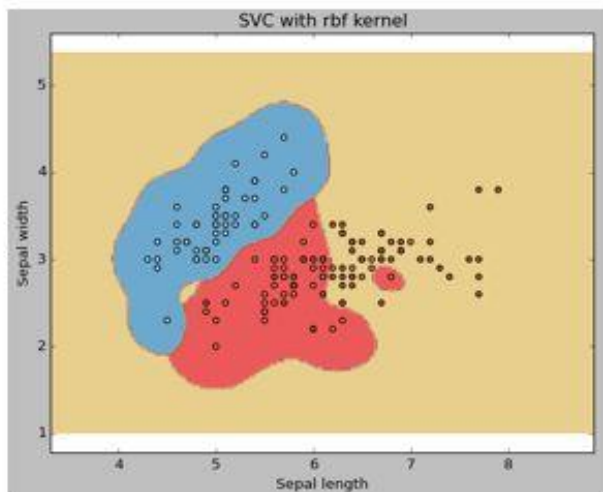
```
svc = svm.SVC (kernel = 'rbf' , C = 1, gamma = 10) .fit (X, y)
```

```
svc = svm.SVC (kernel = 'rbf' , C = 1, gamma = 100) .fit (X, y)
```

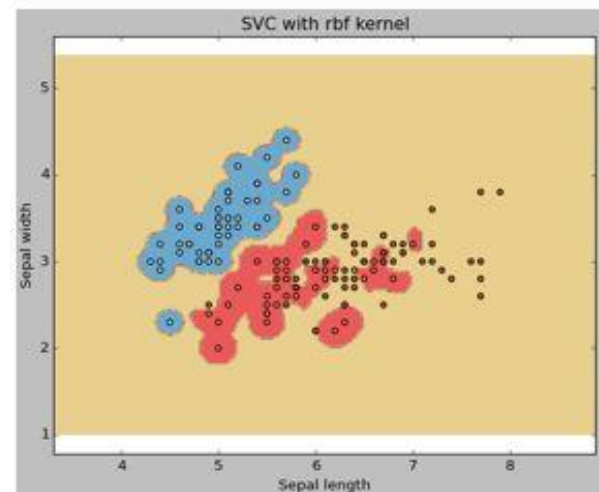
gamma = 0



gamma = 10



gamma = 100



应用例子： 使用RBF的内核

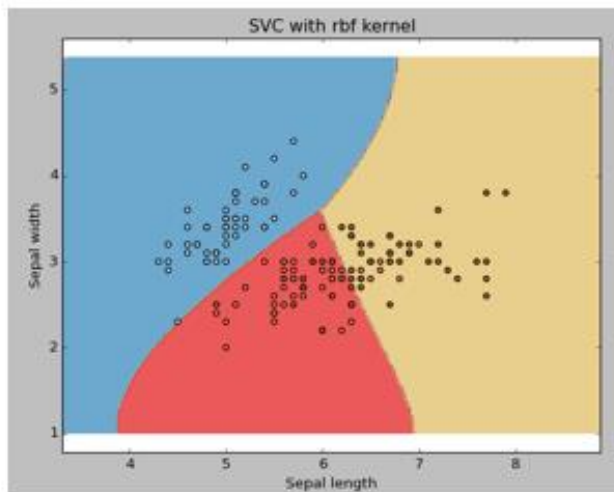
示例：使用不同的C值，如1、100或1000时，

```
svc = svm.SVC (kernel = 'rbf' , C = 1, gamma = 0) .fit (X, y)
```

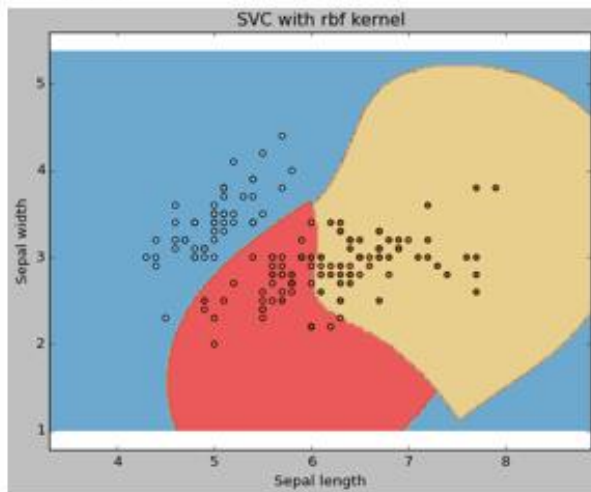
```
svc = svm.SVC (kernel = 'rbf' , C = 100 , gamma = 0) .fit (X, y)
```

```
svc = svm.SVC (kernel = 'rbf' , C = 1000, gamma = 0) .fit (X, y)
```

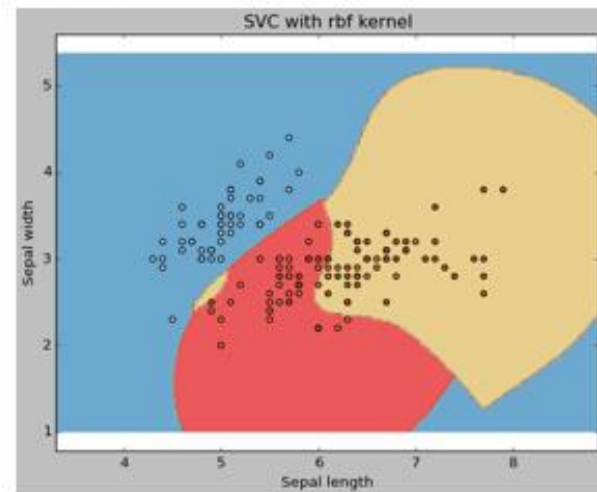
c=1



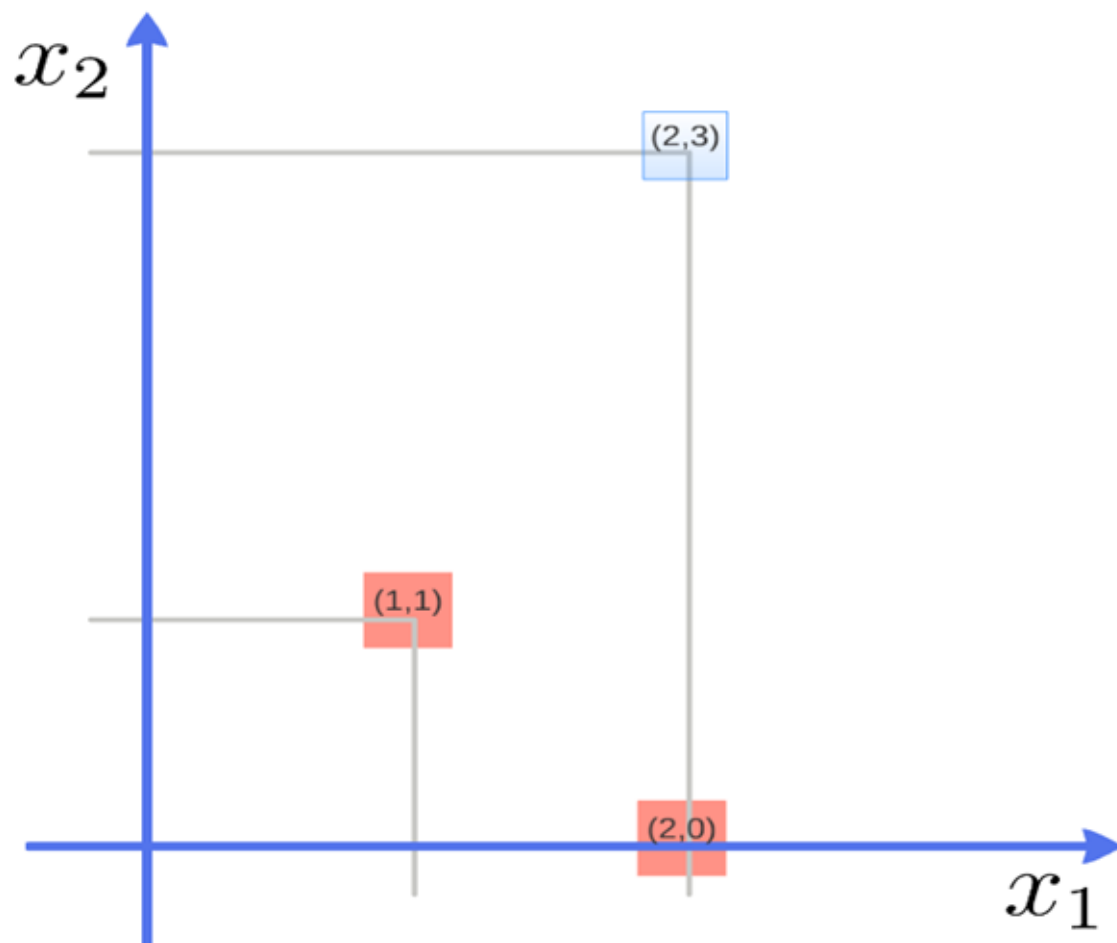
C=100



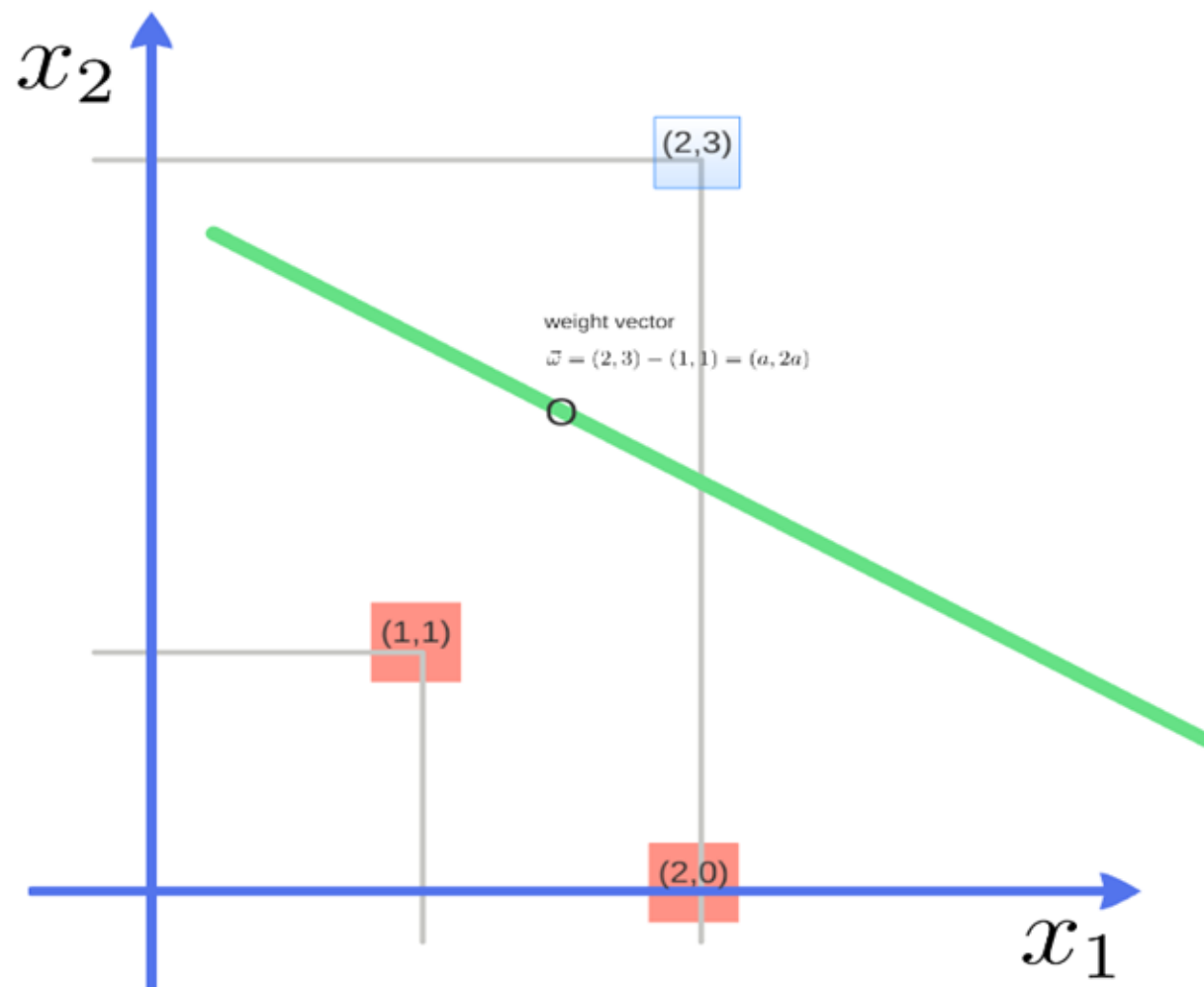
c=1000



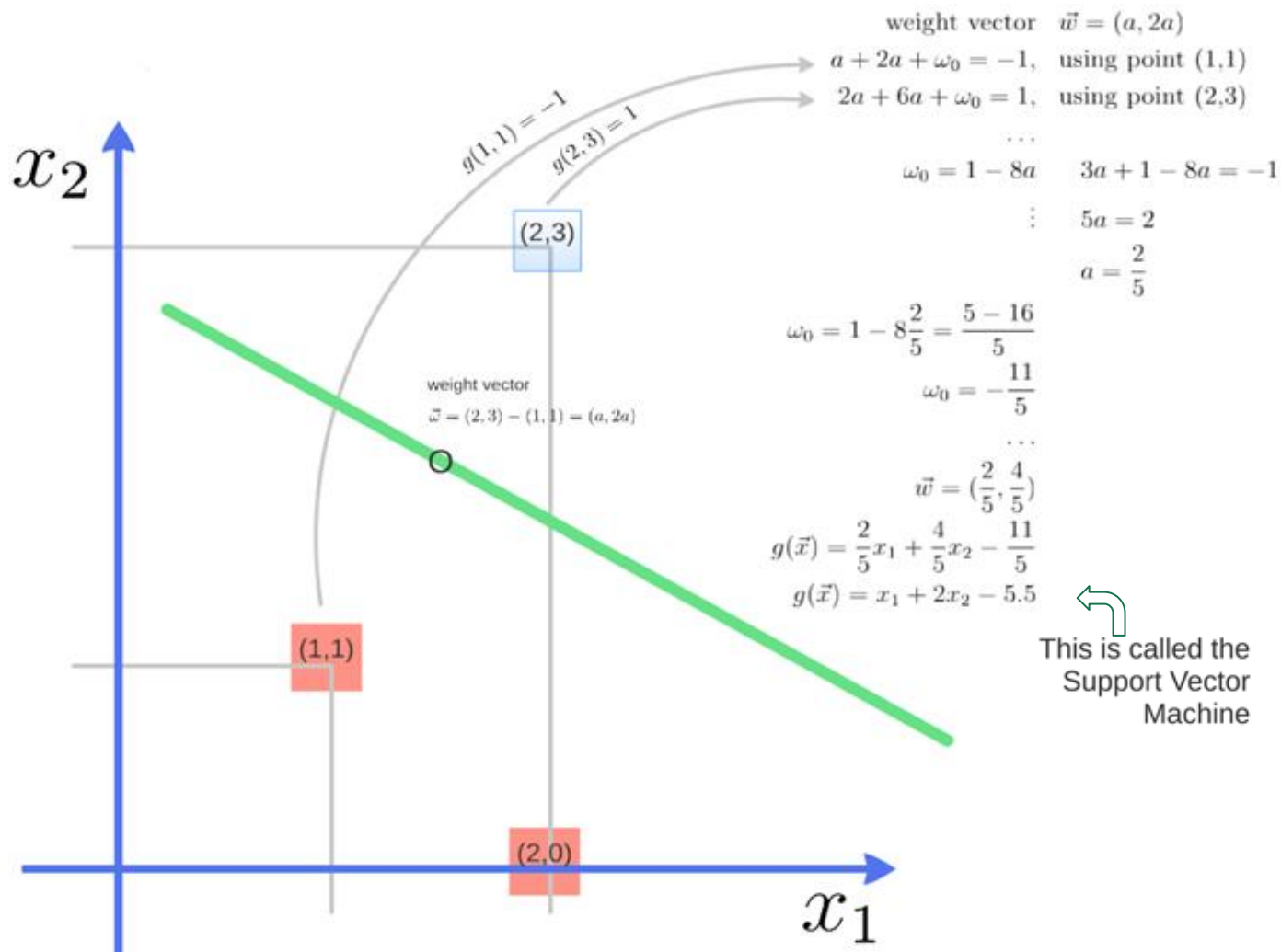
例子



例子



例子



练习题

Class A: $x_1=(3,3)$, $x_2=(4,3)$

Class B: $x_3=(1,1)$,

SVM 分类器的可能决策边界是什么？